The SK game, its optimisation problem and quantum annealing on D-Wave

What the project does, how every algorithm works, what noise does, and how it was tested

Project notes for `Projects_master_HPQC/scripts/QuantumComputing/Dimod`

Contents

1 What this project is **3**

2 The SK game, step by step **3**
- 2.1 Agents, decisions, couplings . . . 3
- 2.2 The network and frustration . . . 4
- 2.3 Local field, satisfaction, energy, reward . . . 5
- 2.4 The single-flip move . . . 6
- 2.5 The landscape: local minima, basins, barriers . . . 6
- 2.6 The game: learning, phases, satisficing . . . 8

3 The algorithms, in the language of the game **9**
- 3.1 Exact enumeration (Gray code) . . . 9
- 3.2 Greedy: steepest descent with restarts . . . 10
- 3.3 The game's learning as an optimiser . . . 11
- 3.4 Simulated annealing . . . 12
- 3.5 Parallel tempering . . . 13
- 3.6 τ-extremal optimisation . . . 14
- 3.7 Tabu search . . . 15
- 3.8 SDP relaxation and Goemans–Williamson rounding . . . 16
- 3.9 The dimod samplers . . . 17
- 3.10 Quantum annealing . . . 18

4 How D-Wave is used in this project **20**
- 4.1 The path of one QPU call . . . 20
- 4.2 Account and access: Leap . . . 20
- 4.3 Functions and parameters, and their role here . . . 20
- 4.4 The back-ends of the campaign . . . 22
- 4.5 Why the QPU budget doubles with two topologies . . . 22

5 Embedding: all-to-all on Zephyr and Pegasus **22**

6 Chains and chain strength: how it was worked out **28**

7 Noise: what it does, what problems it raises, how to reduce it **29**
- 7.1 Random errors on the couplings . . . 30
- 7.2 Chain breaks, and chains that are too rigid . . . 30
- 7.3 Finite effective temperature . . . 31
- 7.4 Systematic biases and spin-reversal transforms . . . 31
- 7.5 Summary . . . 32

8 How everything was tested, and what testing taught **32**
8.1 The testing procedure 32
8.2 What testing found (experience) 33

9 Running it **33**

A Glossary **34**

1 What this project is

The starting point is the model of Garnier-Brun, Benzaquen and Bouchaud, *Unlearnable games and "satisficing" decisions* (arXiv:2312.12252): N agents take binary decisions, every agent's reward depends on all the others through random couplings, and the agents *learn* by remembering past rewards. Inside the game sits an optimisation problem: the configuration of decisions that maximises the average reward. That problem is the Sherrington–Kirkpatrick (SK) spin glass, the textbook example of a hard optimisation landscape.

The project does three things.

1. It **reproduces the game** (package `skgame`, all figures of the paper).
2. It **benchmarks optimisers** on the planner's problem: exact enumeration, greedy, the game's own learning, simulated annealing, parallel tempering, extremal optimisation, tabu search, SDP relaxation, and **quantum annealing on D-Wave**. It does this for growing N and for parameter *regimes* where the optimum is easy or hard to retrieve.
3. It **runs this campaign** on a laptop or on Cineca's Leonardo, with the QPU reached through D-Wave Leap. It compares the two D-Wave topologies, Zephyr (Advantage2) and Pegasus (Advantage), with an all-to-all embedding.

where	what
`unlearnable_games_explanation/skgame`	the model, theory, solvers (`optim/solvers.py`), embeddings (`optim/embedding.py`), annealing back-ends (`optim/backends.py`), regimes (`optim/regimes.py`)
`unlearnable_games_explanation/main_simulation.py`	the staged campaign: `init`, `instances`, `classical`, `embed`, `qpu/sim`, `analyze` (+ notebook)
`unlearnable_games_explanation/local/`	local environment and one-command pipeline (`README_LOCAL.md`)
`unlearnable_games_explanation/cineca/`	Leonardo: offline environment, Slurm jobs, sync with a local machine (`README_CINECA.md`)
`visualization/SK_game`	the model step by step, glossary
`visualization/optimal_algorithms`	every algorithm, with traces of spin state and environment
`visualization/embedding_explanation`	how K_N is embedded on Zephyr and Pegasus; chains and tuning
`visualization/noise_and_chains`	chain strength derivation, noise experiments

Table 1: Map of the project. Every figure in this document is regenerated by the script of its folder.

2 The SK game, step by step

2.1 Agents, decisions, couplings

Each agent $i = 1, \dots, N$ takes a **decision** $S_i = \pm 1$, its **spin**. The reward of agent i is $R_i = \sum_j J_{ij} S_j$. The couplings are random,

$$J = \Big(1 - \frac{\varepsilon}{2}\Big) J^S + \frac{\varepsilon}{2} J^A + \frac{J_0}{N}, \qquad J^S = J^{S\top},\ J^A = -J^{A\top},\ \text{entries} \sim \mathcal{N}(0, 1/N). \tag{1}$$

The **non-reciprocity** ε controls how much J_{ji} resembles J_{ij}; their correlation is $\eta = (1-\varepsilon)/(1-\varepsilon+\varepsilon^2/2)$, equal to 1, 0, -1 at $\varepsilon = 0, 1, 2$. J_0 is a cooperative bias.

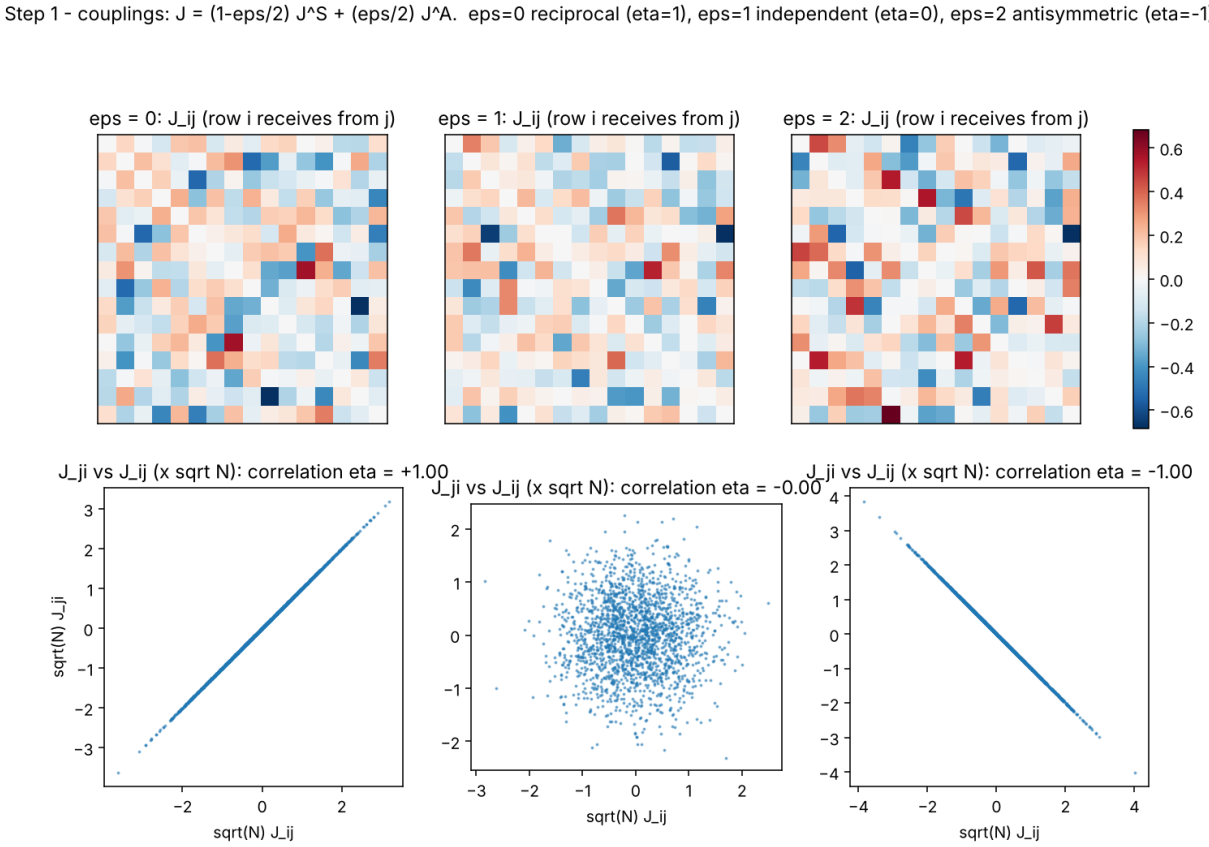


Figure 1: Couplings for $\varepsilon = 0, 1, 2$ and the measured correlation η between J_{ij} and J_{ji}.

2.2 The network and frustration

The planner only sees the symmetric part $W_{ij} = \frac{1}{2}(J_{ij} + J_{ji})$ (zero diagonal). $W_{ij} > 0$: i and j prefer to agree; $W_{ij} < 0$: they prefer to disagree. A loop of couplings whose signs cannot all be satisfied is **frustrated**. In SK about half of all triangles are frustrated, and frustration is the root of the difficulty.

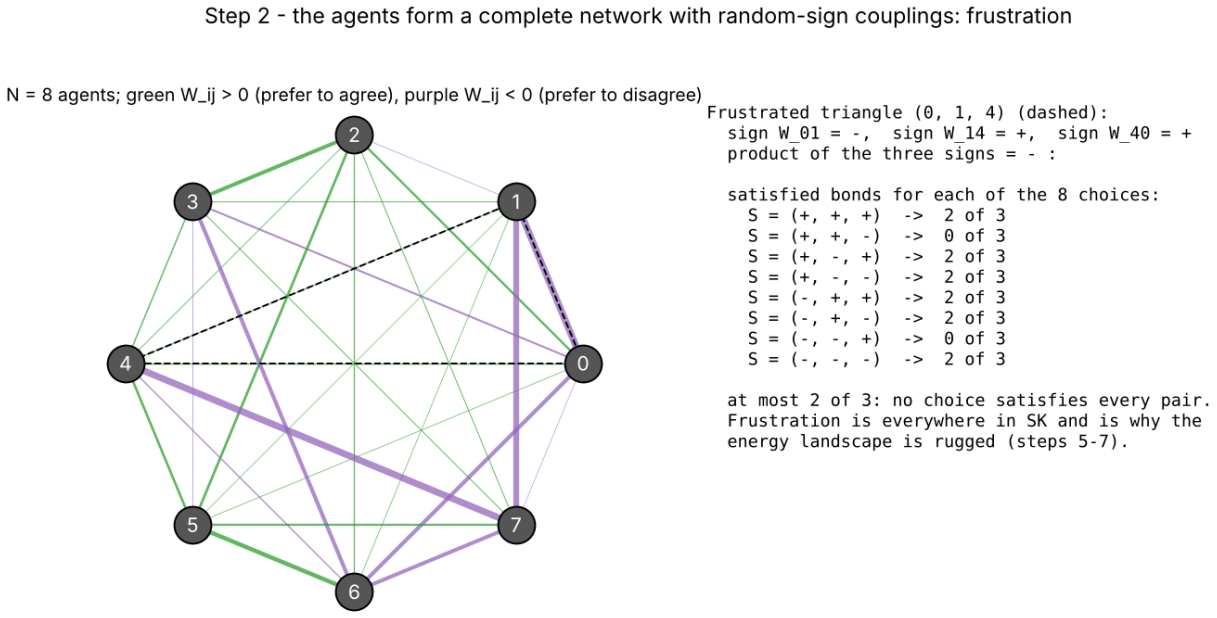


Figure 2: Eight agents. In the dashed triangle no choice satisfies all three pairs: at most 2 of 3.

2.3 Local field, satisfaction, energy, reward

For a configuration S:

$$h_i = \sum_j W_{ij} S_j \qquad \textbf{local field}\text{: the pull of the others on } i, \tag{2}$$

$$\lambda_i = S_i h_i \qquad \textbf{satisfaction}\text{: } > 0 \text{ satisfied, } < 0 \text{ unsatisfied,} \tag{3}$$

$$H(S) = -\sum_{i<j} W_{ij} S_i S_j = -\tfrac{1}{2} \sum_i \lambda_i \qquad \textbf{energy}, \tag{4}$$

$$R_N(S) = \tfrac{1}{N} S^\top J S = \tfrac{1}{N} S^\top W S = -\tfrac{2H}{N} \qquad \textbf{average reward}. \tag{5}$$

Maximising the average reward is minimising the energy: the planner's problem is the SK ground state.

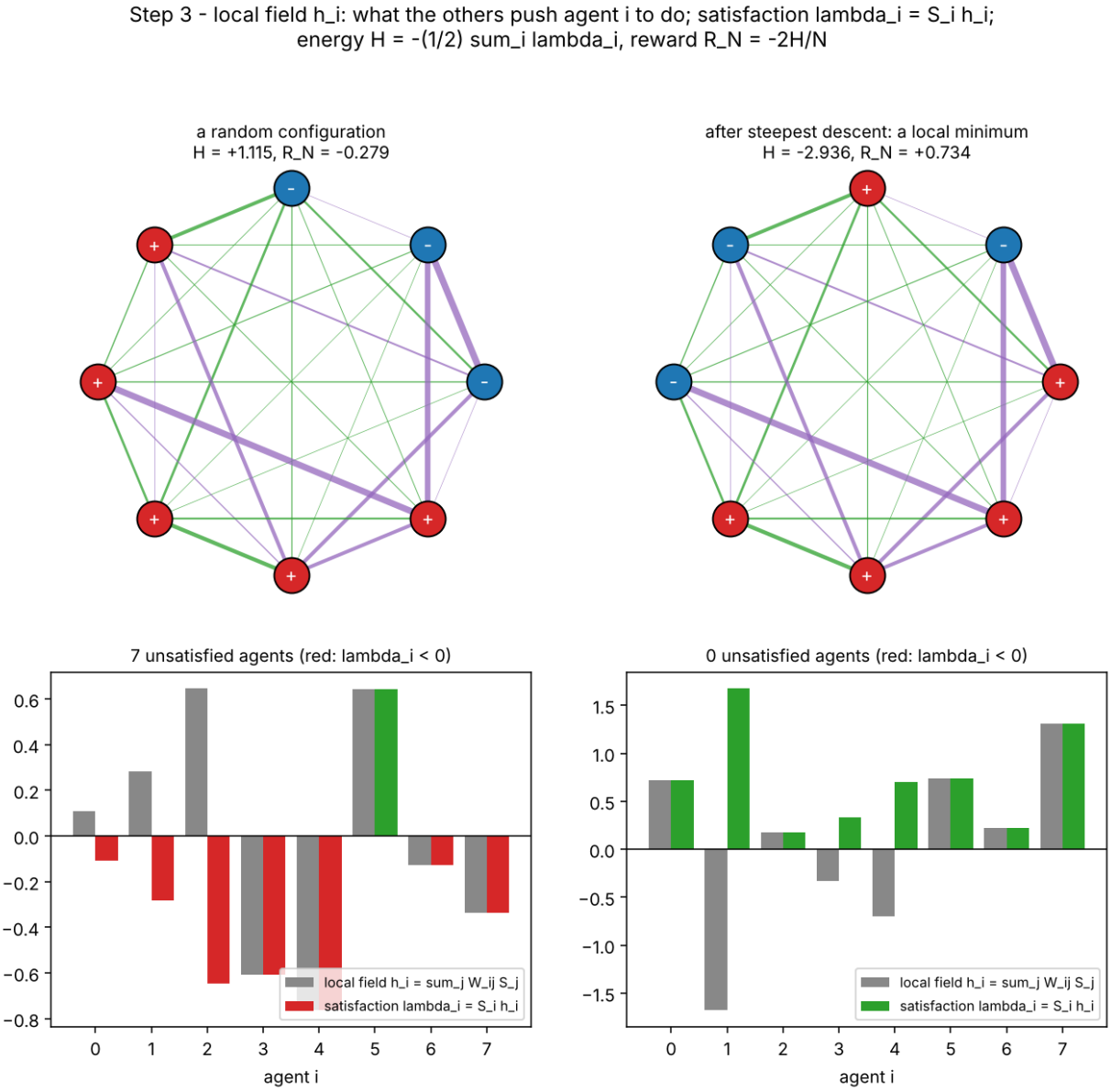


Figure 3: Left: a random configuration with unsatisfied agents (red bars). Right: after flipping unsatisfied agents until none is left: everyone is satisfied and the energy is lower.

2.4 The single-flip move

Flipping agent k changes only the terms that contain k:

$$\Delta H_k = 2\sum_j W_{kj} S_k S_j = 2\lambda_k, \qquad \text{then } h_i \leftarrow h_i + 2W_{ik}S_k^{\text{new}} \quad (O(N)). \tag{6}$$

A flip lowers the energy if and only if the agent is unsatisfied. Almost every classical algorithm below is built from this move.

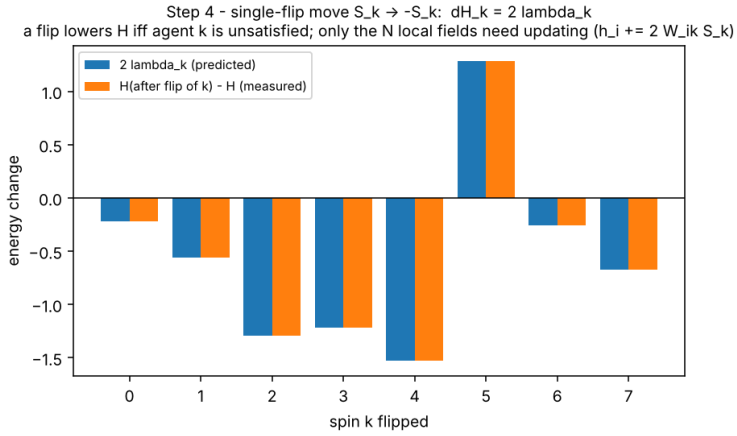


Figure 4: The predicted $2\lambda_k$ and the measured energy change coincide.

2.5 The landscape: local minima, basins, barriers

A **local minimum** (metastable state) is a configuration where every agent is satisfied ($\lambda_i \geq 0$). No single flip improves it, but it need not be the **ground state**.

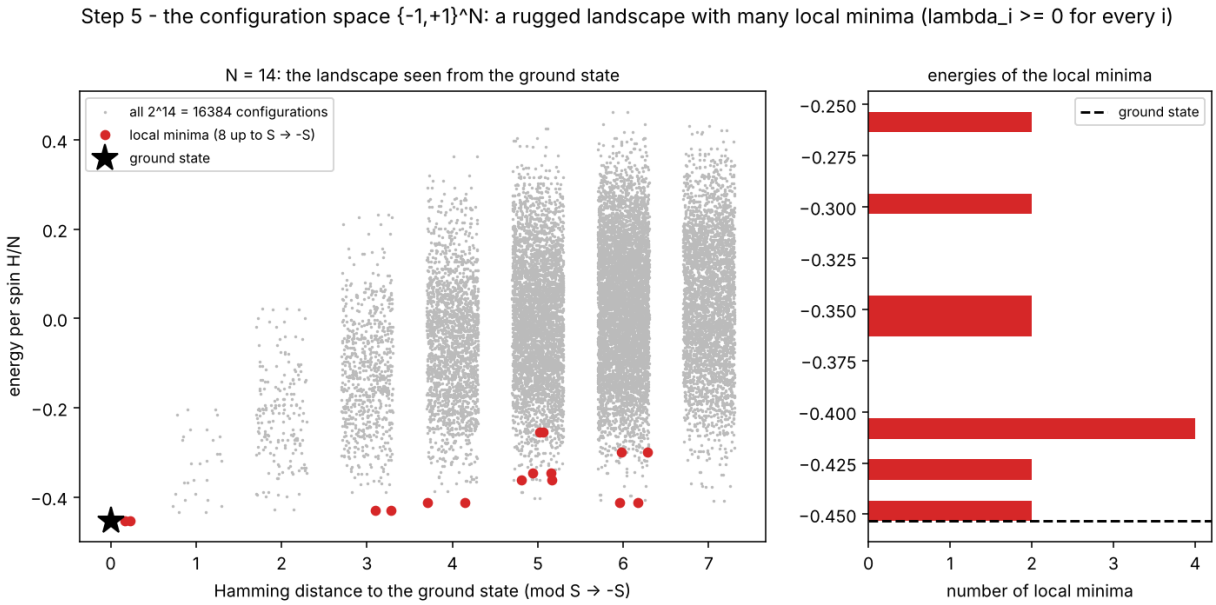


Figure 5: All 2^{14} configurations of one instance. The local minima (red) can be close in energy to the ground state but several flips away: separated by energy barriers.

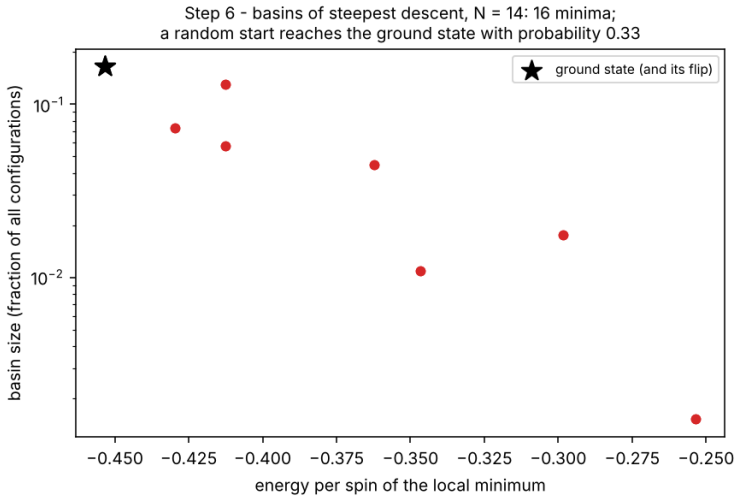


Figure 6: Basins of steepest descent: a random start ends in the ground state with probability 0.33 for $N = 14$.

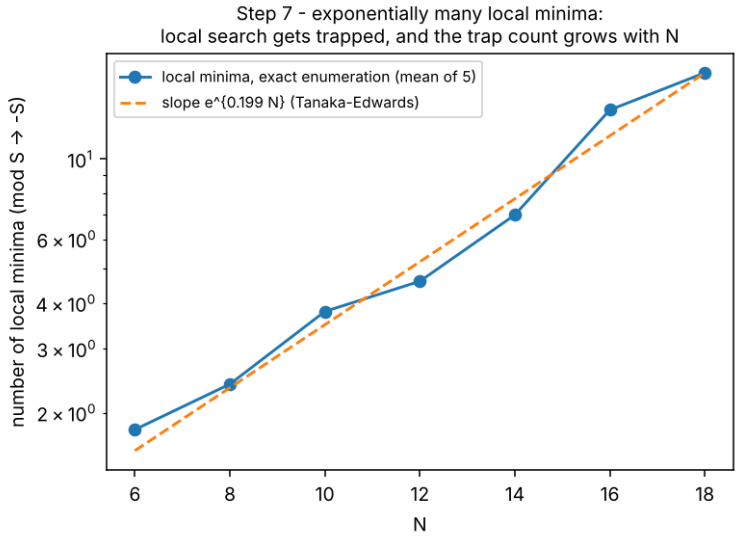


Figure 7: Exact count of local minima: $\propto e^{0.199N}$. The ground state's basin becomes an exponentially small fraction of the space.

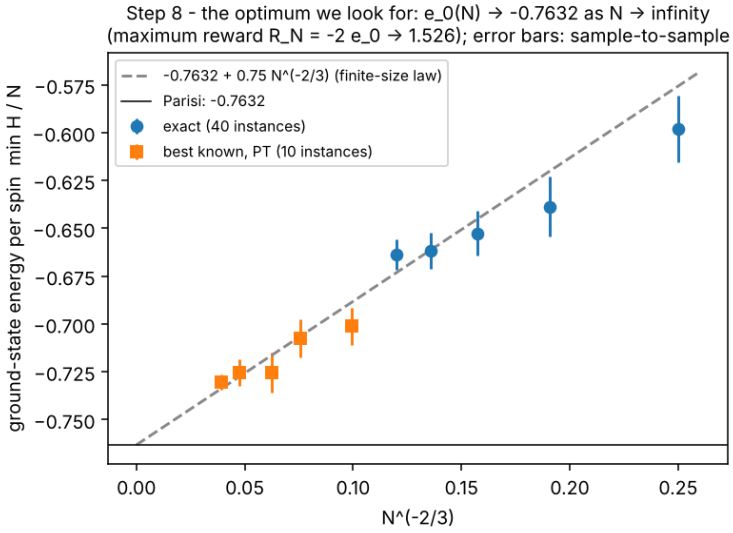


Figure 8: The ground-state energy per spin tends to the Parisi value -0.7632, with corrections $\approx 0.75N^{-2/3}$.

2.6 The game: learning, phases, satisficing

Each agent keeps an **incentive** Q_i and plays $S_i = \operatorname{sign} Q_i$ (at temperature $T = 1/\beta$: $P[S_i = \pm 1] = \frac{1}{2}(1 \pm \tanh \beta Q_i)$), then

$$Q_i(t+1) = (1-\alpha)Q_i(t) + \alpha R_i(t). \tag{7}$$

Depending on $(\alpha, \beta, \varepsilon, J_0)$ the long-time behaviour is a fixed point, a 2-cycle, chaos, or a paramagnet. These are the regimes of the campaign.

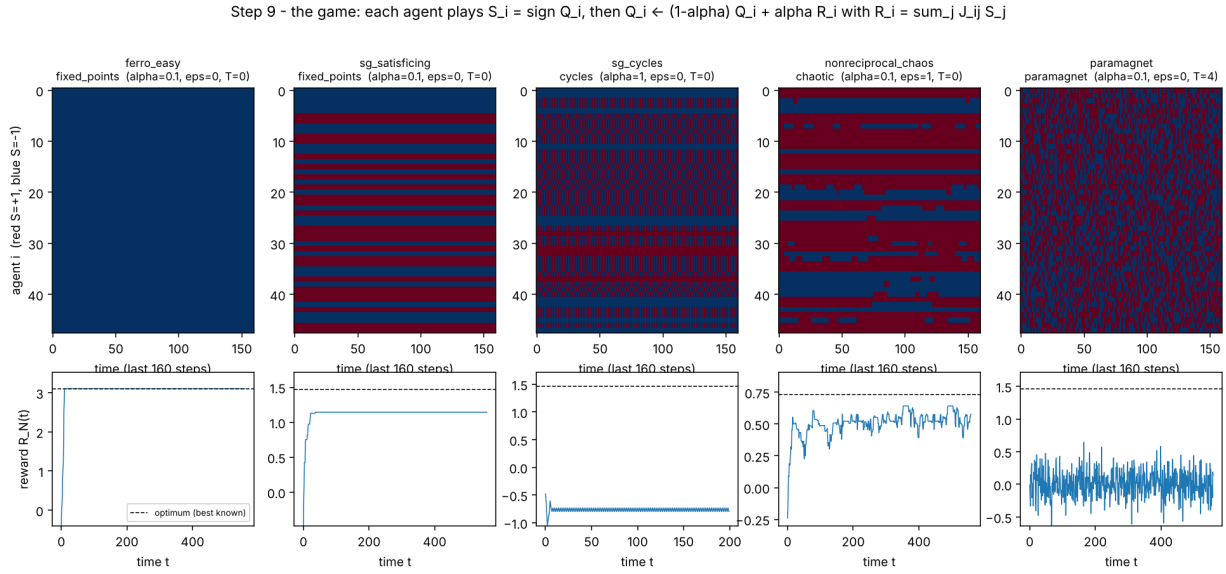


Figure 9: The five regimes used in the campaign. Top: decisions over time; bottom: reward vs the optimum.

At a fixed point $Q_i = R_i$ and $S_i = \operatorname{sign} R_i$, so $S_i R_i \geq 0$. At $\varepsilon = 0$, where $R_i = h_i$, **every fixed point is a local minimum**. The game satisfies everyone, but it does not optimise: this is *satisficing*.

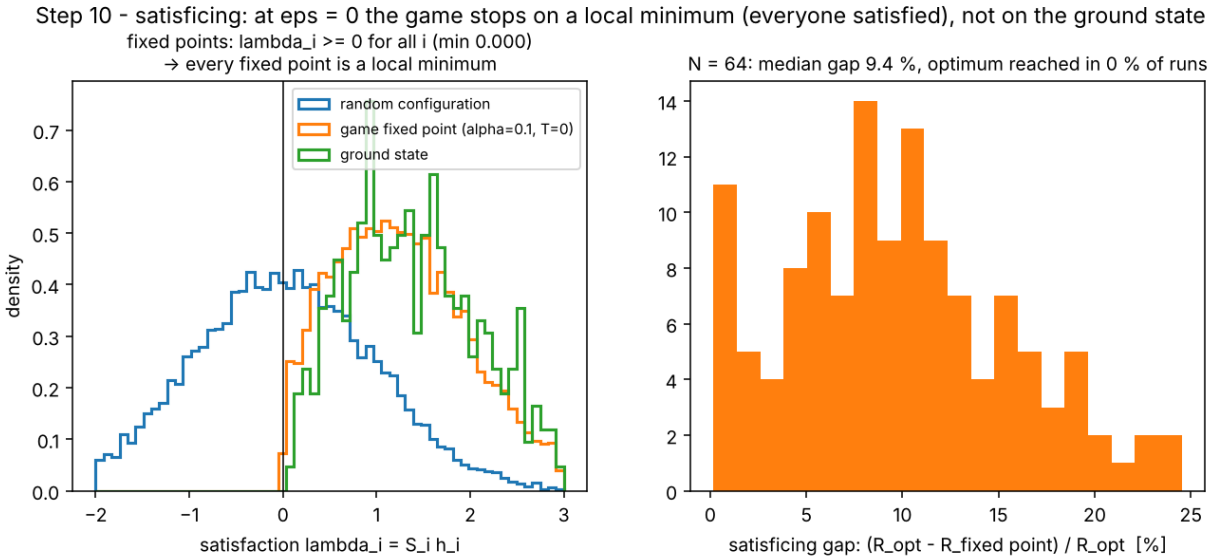


Figure 10: At the game's fixed points no agent is unsatisfied; the median distance from the optimum is 9.4% at $N = 64$.

regime	$(\varepsilon, J_0, \alpha, \beta)$	planner problem	game phase	expected
`ferro_easy`	$(0, 3, 0.1, \infty)$	ferromagnetic	fixed point	easy for everybody
`sg_boundary`	$(0, 1, 0.1, \infty)$	SG/ferro boundary	fixed point	intermediate
`sg_satisficing`	$(0, 0, 0.1, \infty)$	spin glass	fixed point	hard; game below optimum
`sg_cycles`	$(0, 0, 1, \infty)$	spin glass	2-cycle	game never optimal
`nonreciprocal_chaos`	$(1, 0, 0.1, \infty)$	spin glass	chaos	game never optimal
`paramagnet`	$(0, 0, 0.1, 0.25)$	spin glass	paramagnet	random play

Table 2: Regimes. The static difficulty depends only on $J_0^{\text{eff}} = J_0/(1 - \varepsilon/2)$: spin glass below 1, ferromagnet above.

3 The algorithms, in the language of the game

Every algorithm answers one question: *how do we get out of a local minimum?* For each one we give what it does to the spins, how it escapes, its cost, where it is in the code, and two kinds of pictures:

- the **spin state and environment** figure (prefix `se_`). Three rows share the time axis: the spin state (red = the agent agrees with the ground state $S^\star$, one global sign per run), the *environment* the algorithm reacts to, and the energy;
- the **behaviour** figure (prefix `alg`): statistics over many runs.

The traces come from `optimal_algorithms/traced.py`, which has the same update rules as the benchmark solvers. Steepest descent reproduces the package's `_greedy` exactly; the others reach the exact optimum on small instances.

3.1 Exact enumeration (Gray code)

It visits all 2^{N-1} configurations (S_{N-1} is fixed by the global flip symmetry) in an order where consecutive configurations differ by one flip, so each step is a single-flip move. The cost is $N2^{N-1}$ operations: $24 \cdot 2^{23} \approx 2.0 \times 10^8$ for $N = 24$ (sub-second), $64 \cdot 2^{63} \approx 5.9 \times 10^{20}$ for $N = 64$ (impossible). This is why the exact reference stops at $N = 24$.

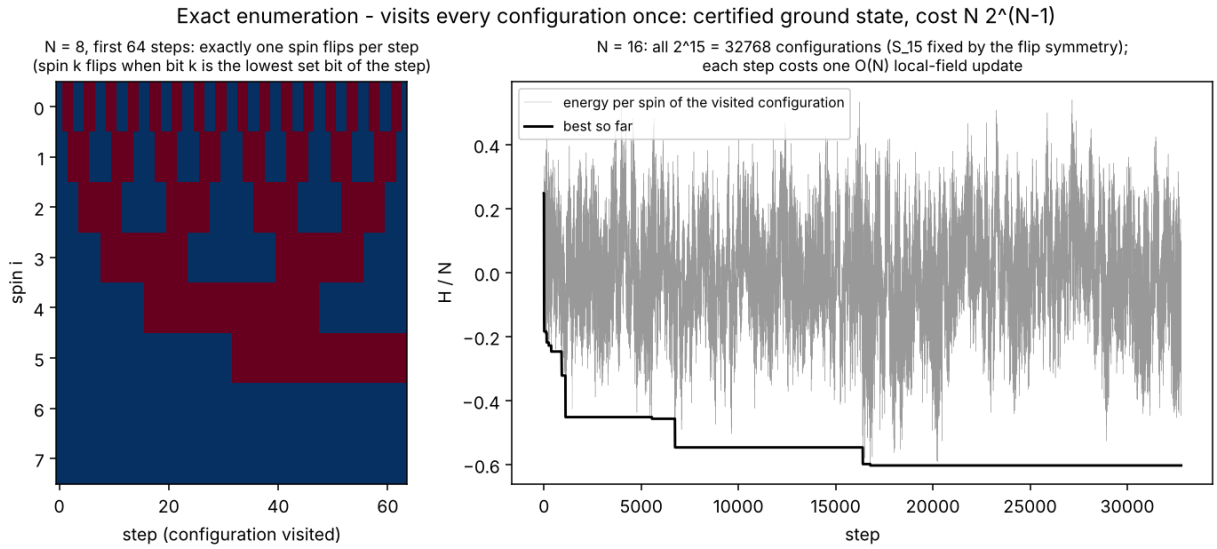


Figure 11: Left: one spin changes per step. Right: all 2^{15} energies for $N = 16$, and the best so far.

3.2 Greedy: steepest descent with restarts

It flips the most unsatisfied agent until none is left, then restarts from a new random configuration. It escapes only by restarting: every descent ends in the basin it started in.

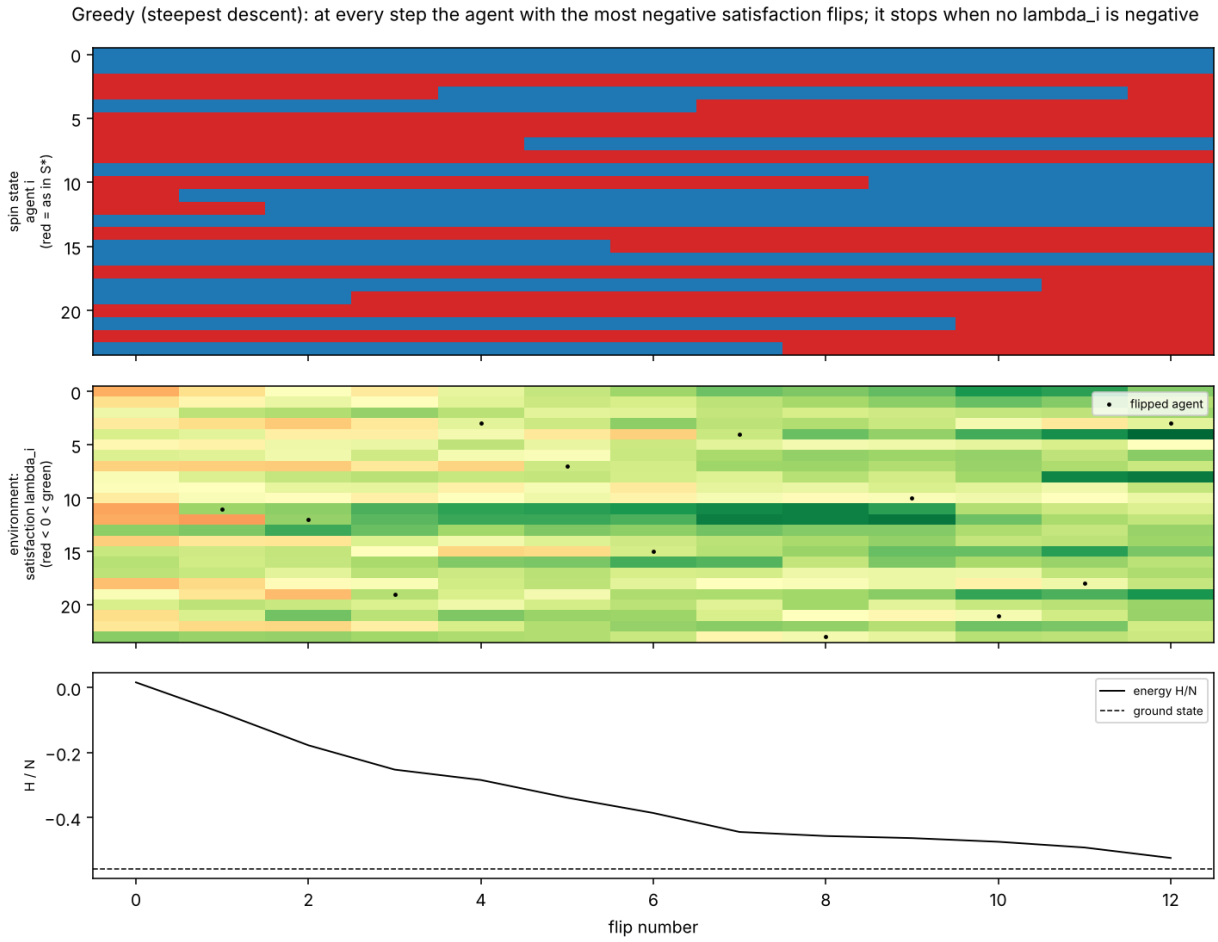


Figure 12: Spin state, satisfactions (environment) and energy during one descent. The final state is satisfied everywhere and close in energy to $S^\star$, but many agents (blue) still disagree with it.

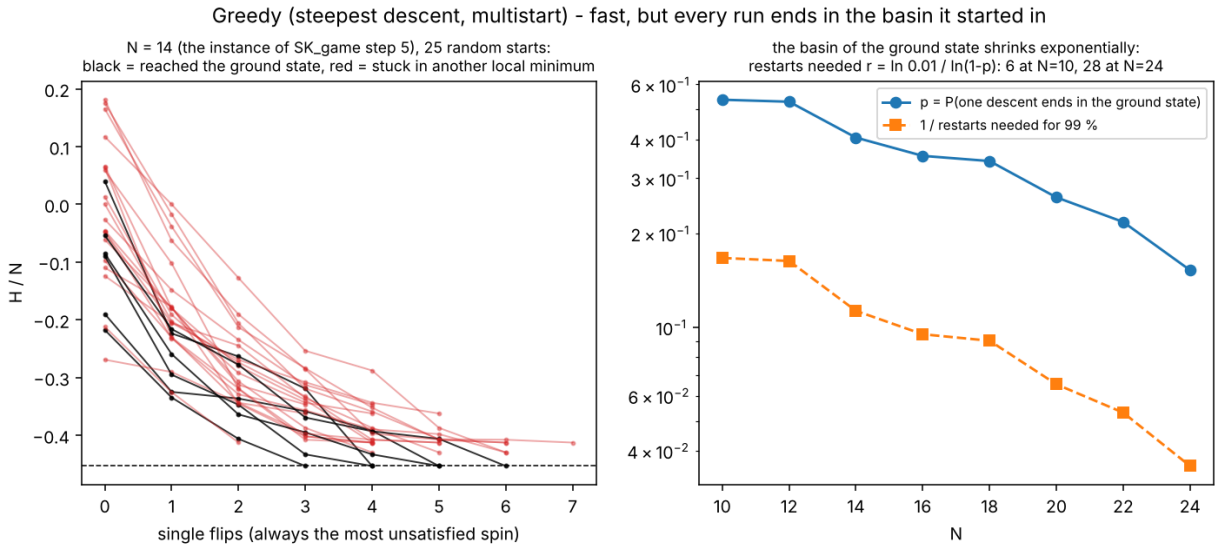


Figure 13: The probability that a descent ends in the ground state falls from 0.52 ($N = 10$) to 0.15 ($N = 24$). The restarts needed for 99% confidence, $r = \ln 0.01 / \ln(1 - p)$, grow from 6 to 28.

3.3 The game's learning as an optimiser

All agents update $Q_i \leftarrow (1 - \alpha)Q_i + \alpha h_i$ together and play $\operatorname{sign} Q_i$. An agent changes decision once it has been unsatisfied long enough for Q_i to change sign. The run freezes in the first local minimum: a local search with memory.

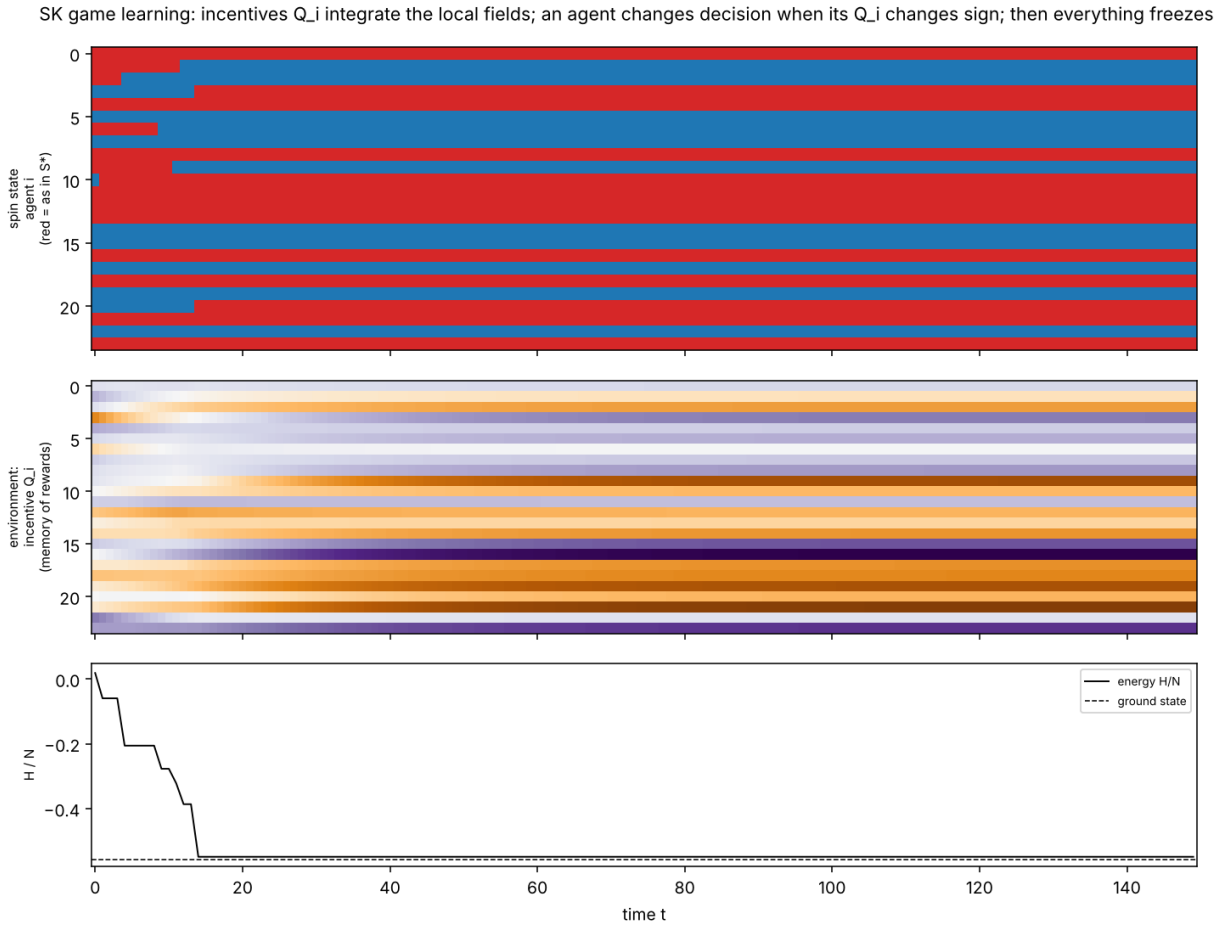


Figure 14: The environment is the memory Q_i. Decisions stop changing after about 15 steps. The final energy is within 1.4% of the ground state, yet 11 of 24 agents disagree with $S^\star$: close in energy, far in configuration.

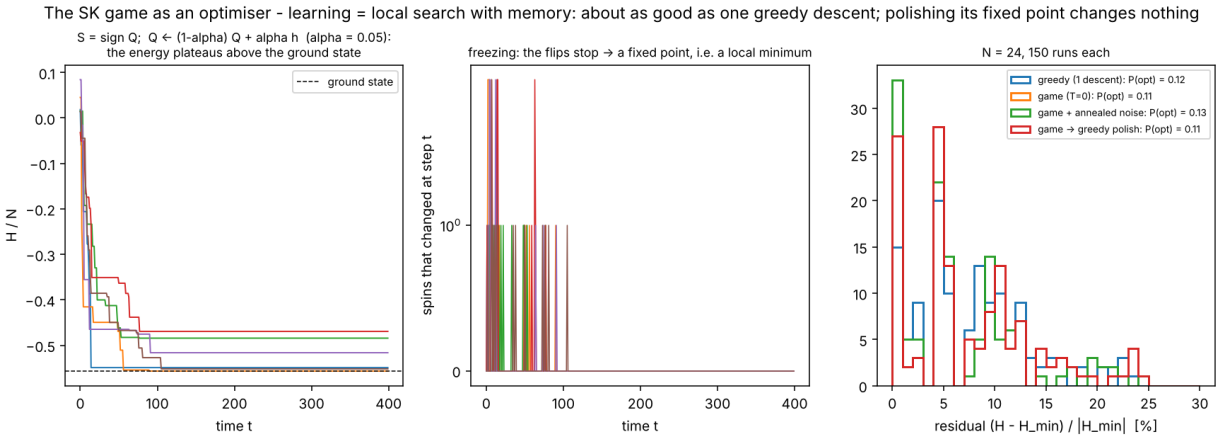


Figure 15: One game run finds the optimum with probability 0.11, one greedy descent 0.12. Greedy polishing of the game's fixed point changes nothing, because it is already a local minimum.

3.4 Simulated annealing

Metropolis single flips: an unsatisfied agent always flips, a satisfied one with probability $e^{-\beta\Delta H} = e^{-2\beta\lambda_k}$. The inverse temperature β rises from 0.1 to 5. When hot it crosses barriers; as it cools, the barriers close one after another. The solver returns the best state seen, polished by greedy.

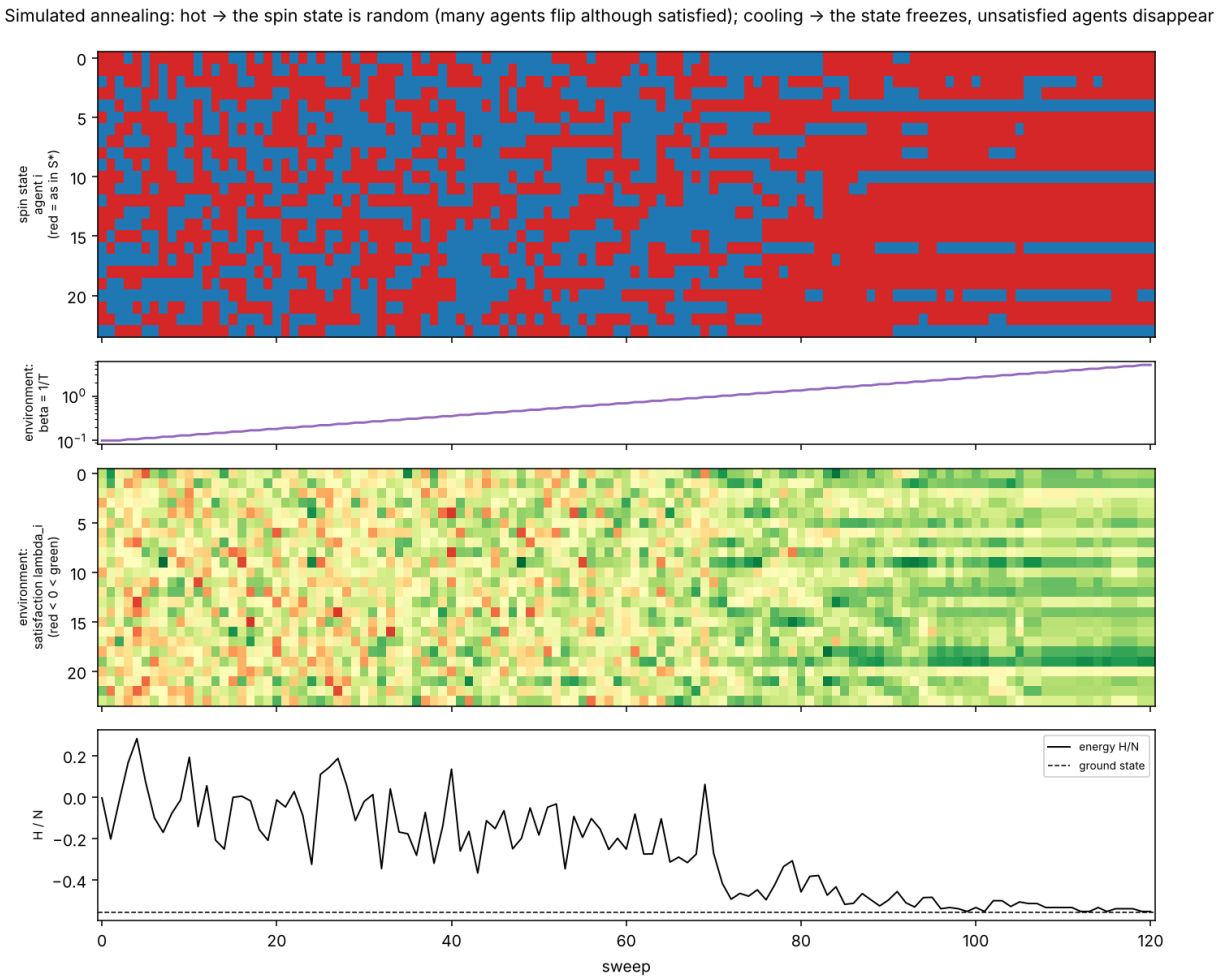


Figure 16: Environment: $\beta(t)$ and the satisfactions. Hot: a random mosaic of decisions. Cooling: groups of agents lock in and the satisfactions turn green.

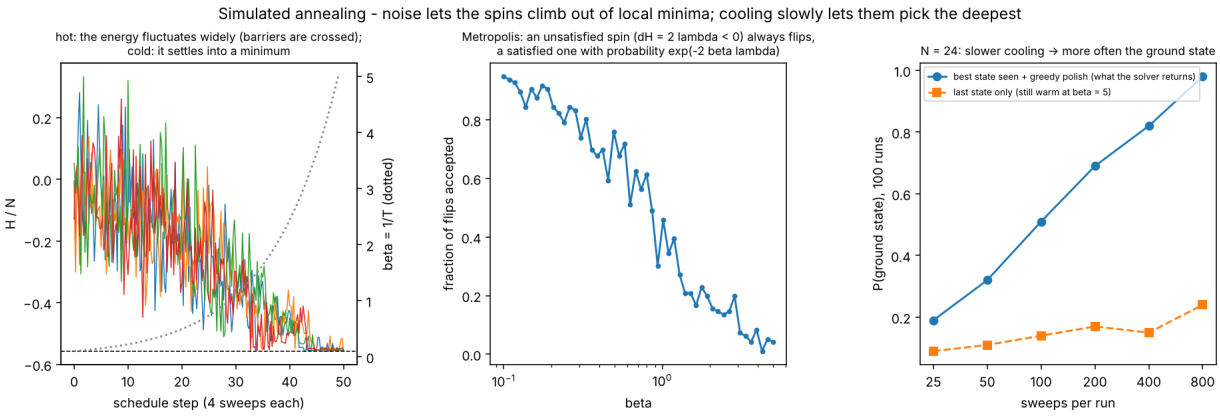


Figure 17: Acceptance falls from 0.95 to 0.05 with β. More sweeps give the ground state more often (0.19 at 25 sweeps, 0.98 at 800). The last state alone is a poor estimate (dashed).

3.5 Parallel tempering

R replicas at temperatures $\beta_1 < \cdots < \beta_R$. After each Metropolis sweep, neighbouring temperatures swap configurations with probability $\min\{1, e^{(\beta_k - \beta_{k+1})(E_k - E_{k+1})}\}$. A trapped replica gets heated, melts out of its minimum and cools into another one. Parallel tempering is also the reference solver above $N = 24$.

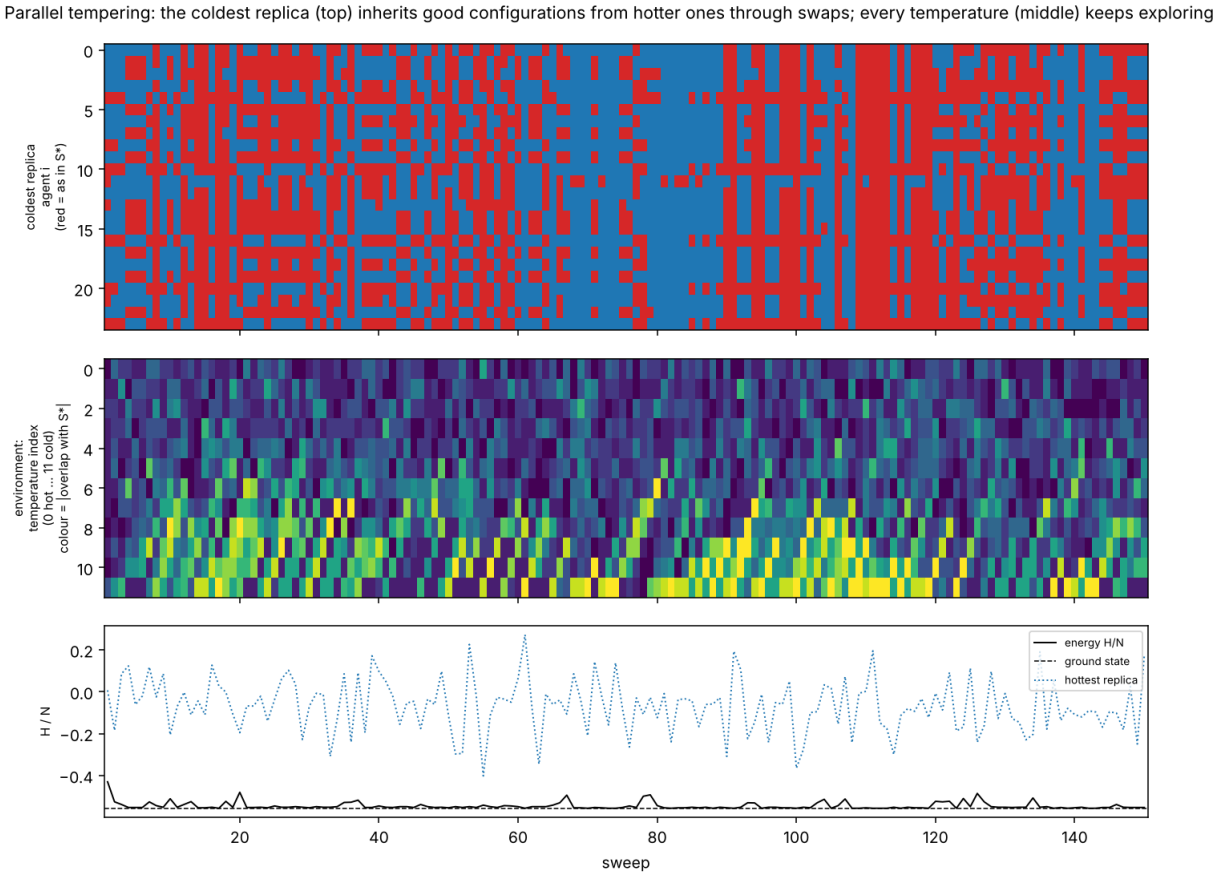


Figure 18: Spin state of the coldest replica, and the environment: overlap with $S^\star$ of the replica at each temperature. Good configurations appear at cold temperatures and are passed down the ladder.

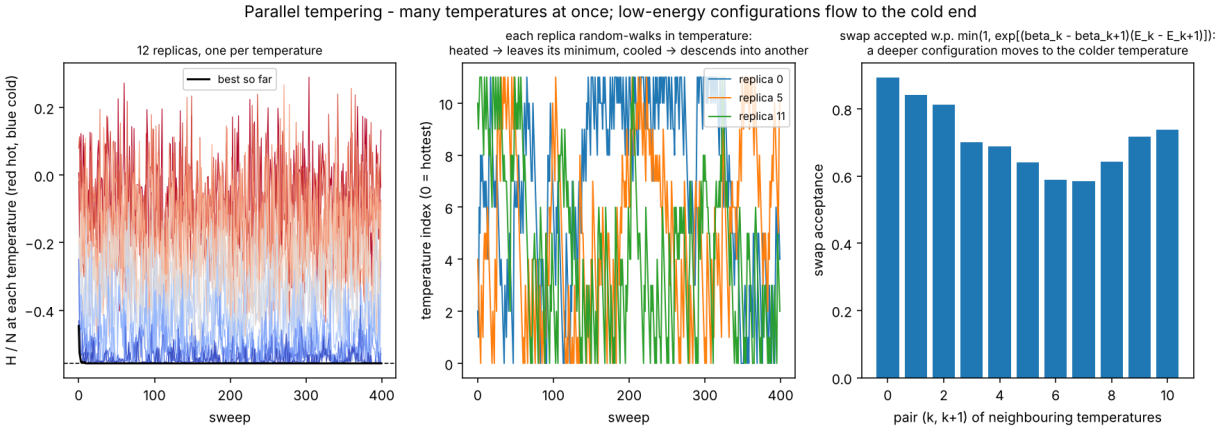


Figure 19: Energies at 12 temperatures; replicas random-walk across the ladder; swap rates 0.6–0.9.

3.6 τ-extremal optimisation

Rank the agents by satisfaction (rank 1 = least satisfied). Pick rank k with probability $\propto k^{-\tau}$ and flip it *unconditionally*. There is no temperature and no acceptance test, so the state never freezes; the best visit is kept.

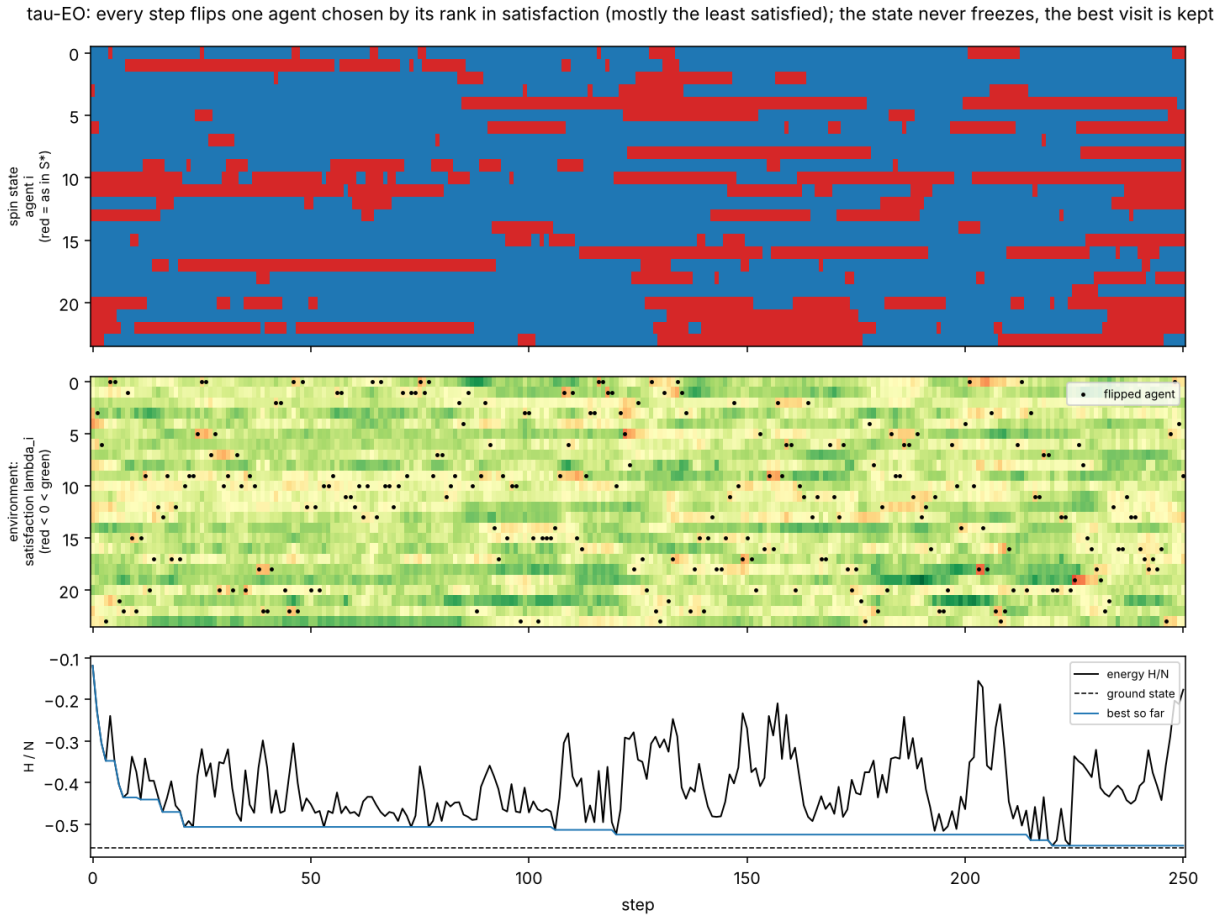


Figure 20: Black dots: the agent flipped at each step, usually among the least satisfied.

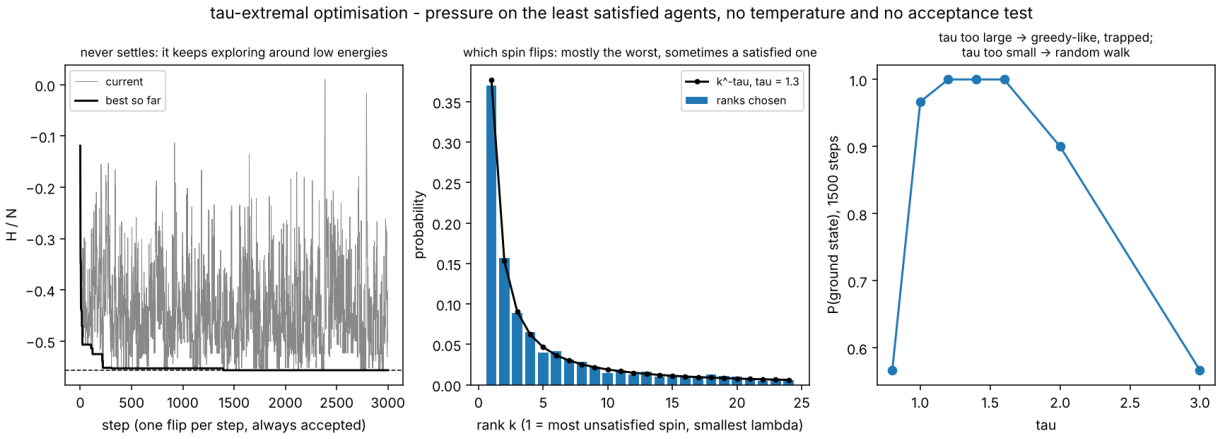


Figure 21: Chosen ranks follow $k^{-1.3}$. The best τ is around 1.2–1.6.

3.7 Tabu search

At every step flip the allowed agent with the smallest $\Delta H = 2\lambda_k$, even uphill. A flipped agent becomes *tabu* for `tenure` steps, so the search cannot fall back into the minimum it just left. D-Wave's `TabuSampler` runs the MST2 multistart variant with tenure $\min(20, \lfloor N/4 \rfloor)$.

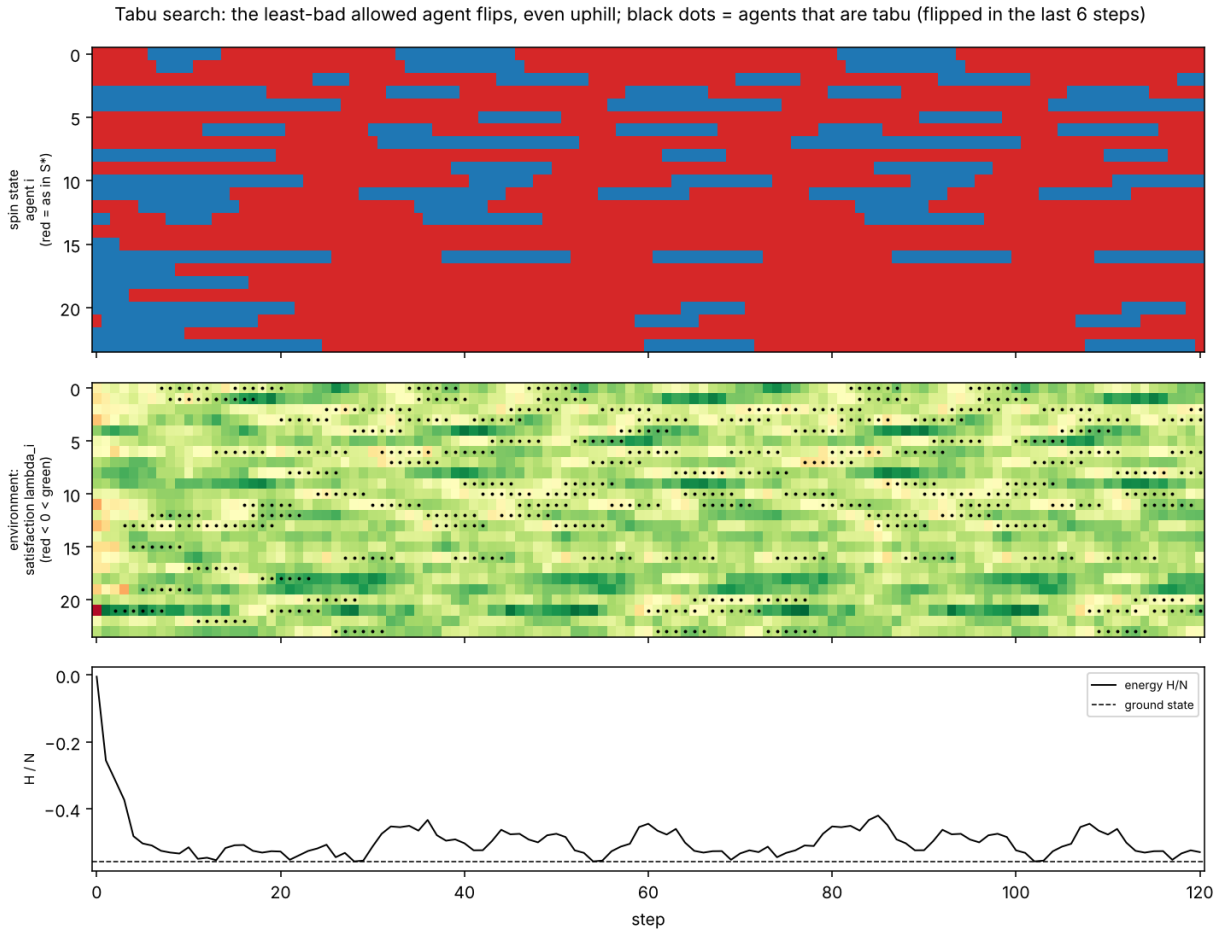


Figure 22: Black dots: agents that are tabu. The energy keeps oscillating close to the ground state, which it visits repeatedly.

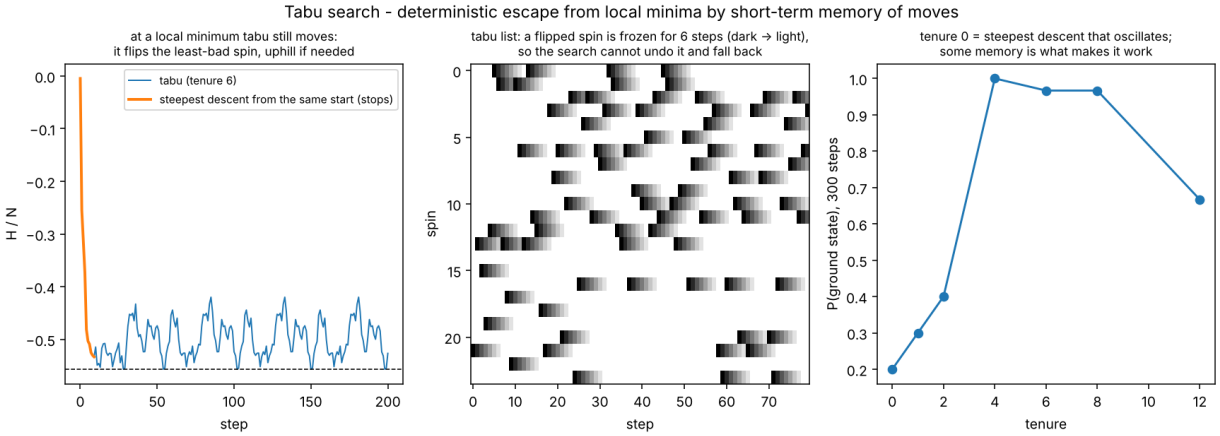


Figure 23: Without memory (tenure 0) it finds the ground state in 20% of runs; with tenure 4–8, in 97–100%.

3.8 SDP relaxation and Goemans–Williamson rounding

Replace S_i by unit vectors $v_i \in \mathbb{R}^k$ and minimise $-\frac{1}{2}\sum W_{ij} v_i \cdot v_j$, which is convex in $X = VV^\top$. The mixing method aligns each vector with its vector local field, $v_i \leftarrow \text{normalise}(\sum_j W_{ij} v_j)$. Rounding with a random hyperplane gives $S_i = \text{sign}(v_i \cdot r)$. The relaxed minimum is a **certified lower bound** on $H_{\min}$.

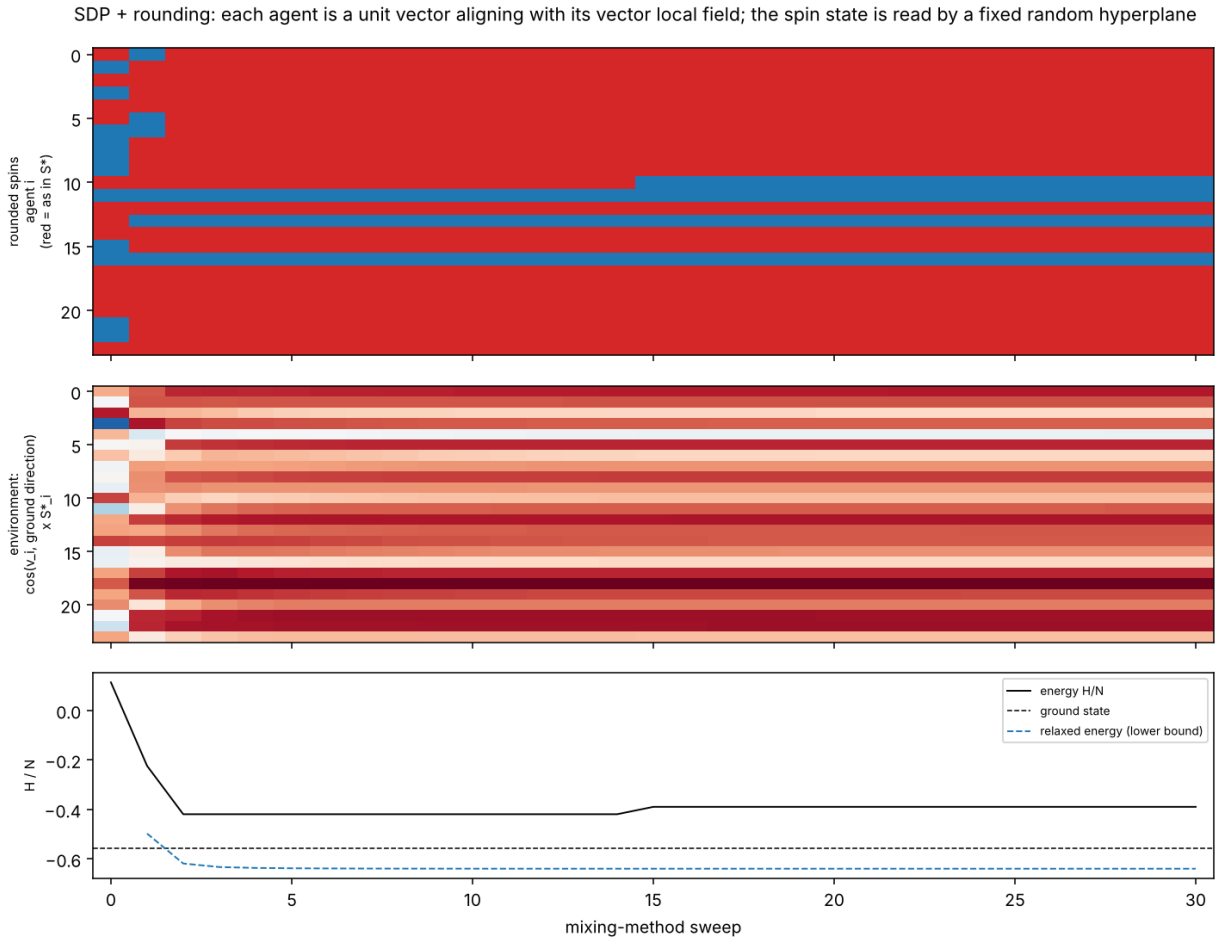


Figure 24: Environment: alignment of each vector with the ground-state direction. The relaxed energy settles below the ground state (the bound).

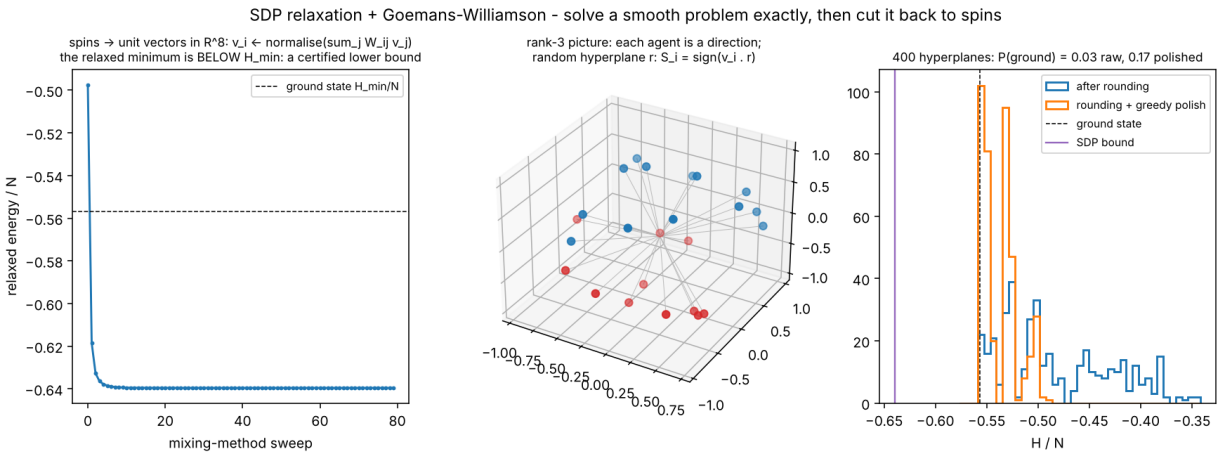


Figure 25: The bound $-0.64N$ lies below $H_{\min} = -0.557N$. Rounding alone hits the optimum 3% of the time; with greedy polishing, 17%.

3.9 The dimod samplers

`dwave.samplers` implements SA (C++, sweeps in fixed order, β range computed from the instance), tabu and steepest descent behind the same `dimod` interface as the QPU.

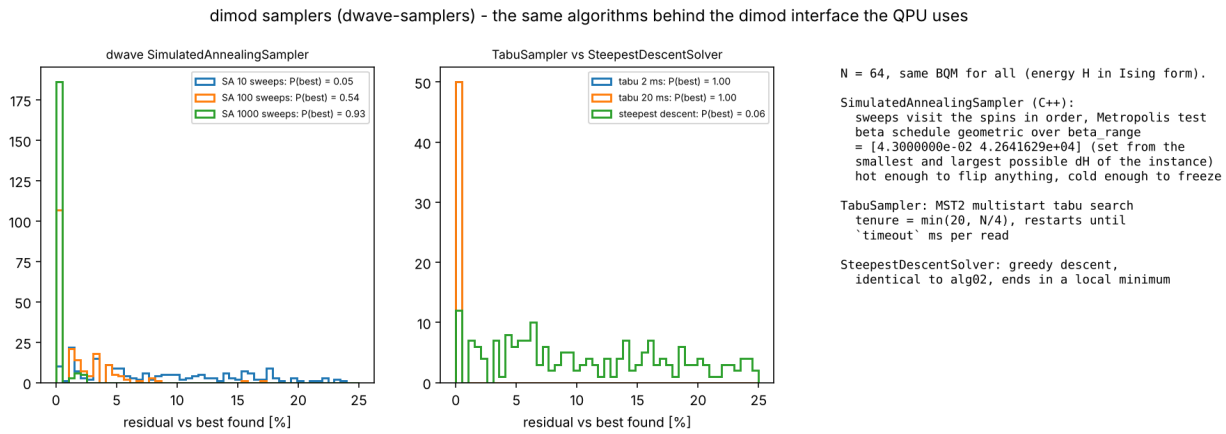


Figure 26: $N = 64$: SA finds the best known energy in 5/54/93% of reads with 10/100/1000 sweeps; tabu in every read.

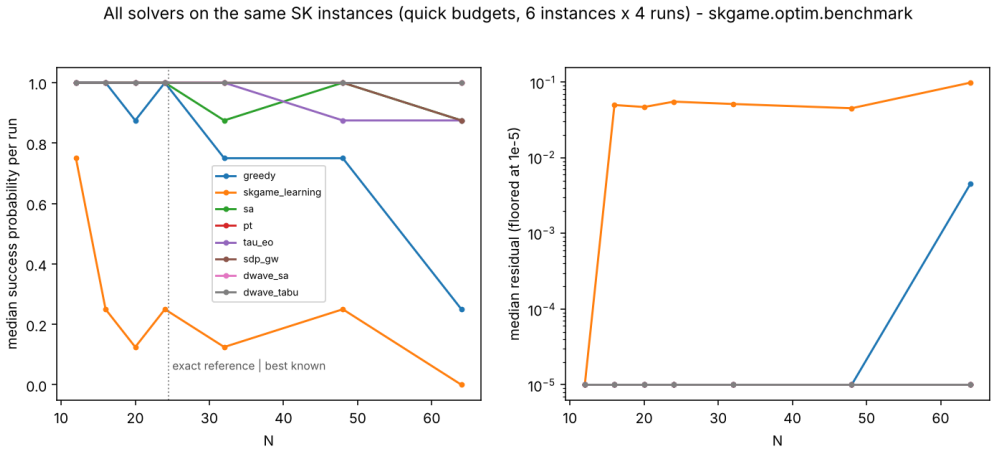


Figure 27: All classical solvers on the same instances (quick budgets). PT, dwave SA and tabu stay at 1 up to $N = 64$; greedy and the game's learning degrade first.

3.10 Quantum annealing

The spins become qubits. The machine starts in the ground state of a transverse field, where every configuration is superposed, and slowly turns it off while turning on the SK energy:

$$\mathcal{H}(s) = -A(s)\sum_i \sigma_i^x + B(s)H_P(\sigma^z), \qquad s = t/T : 0 \to 1. \tag{8}$$

Measuring at $s = 1$ gives one configuration (a *read*). If $T \gg 1/\Delta_{\min}^2$, where $\Delta_{\min}$ is the minimum gap of $\mathcal{H}(s)$, the system stays in its ground state (adiabatic theorem). Instead of climbing barriers, it *tunnels* through them.

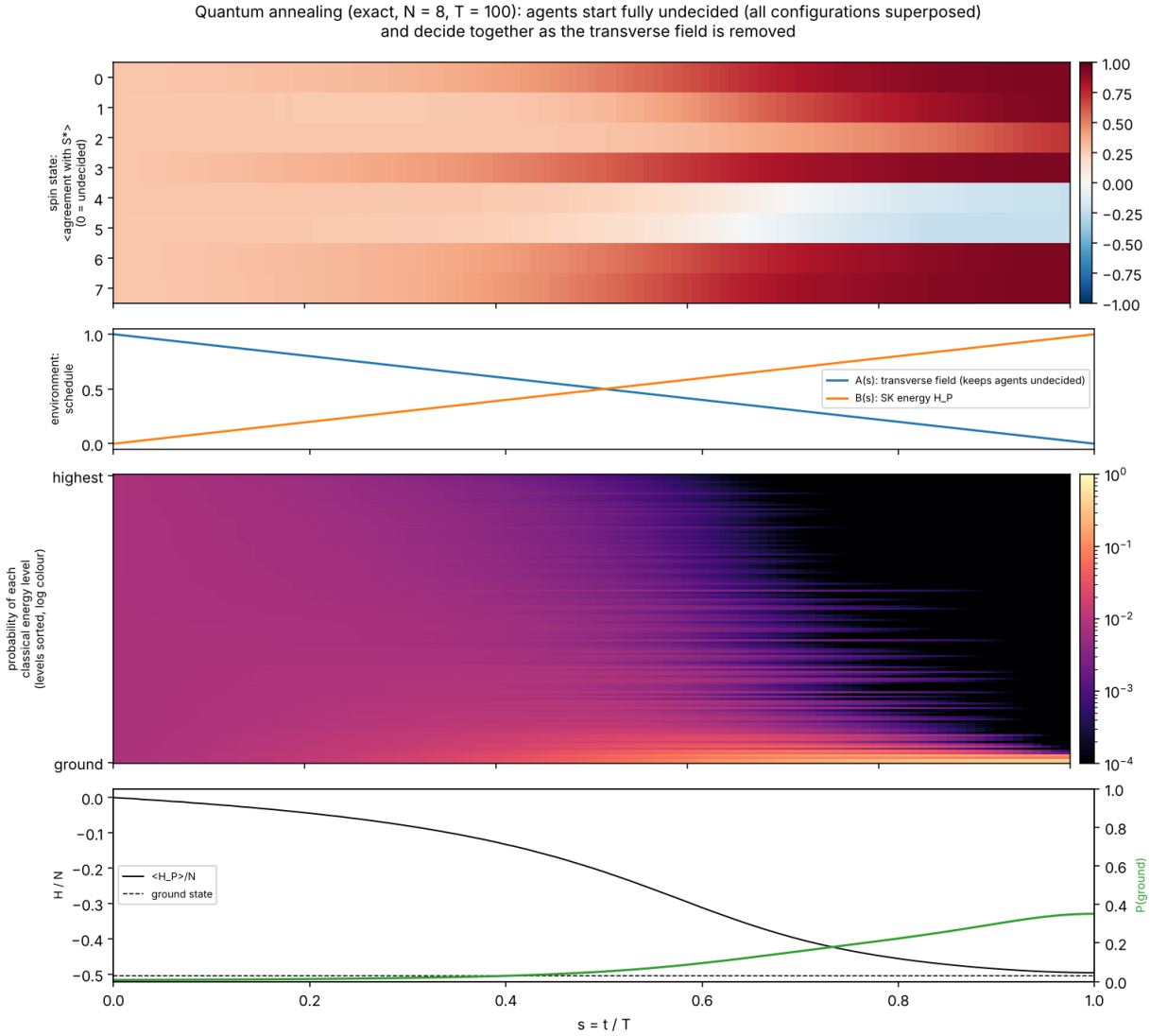


Figure 28: Exact Schrödinger simulation, 8 qubits, idealised schedule $A = 1 - s$, $B = s$. Spin state: the expected agreement of each agent with $S^\star$ (0 = undecided). The agents start undecided and decide together. Probability flows from all levels to the lowest ones. With $T = 100 < 1/\Delta^2_{\min} \approx 700$, two agents often end on the wrong side and $P(\text{ground}) = 0.35$.

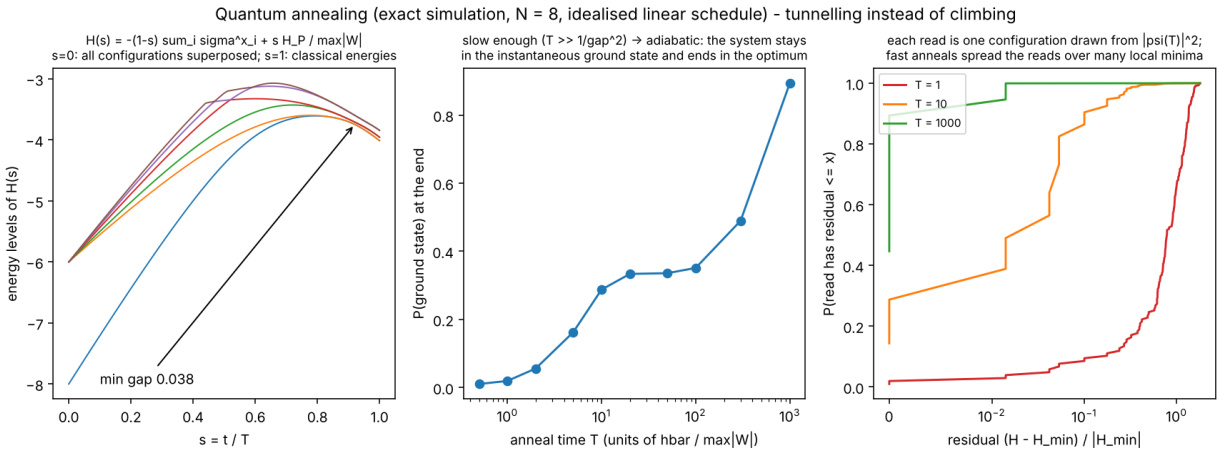


Figure 29: Spectrum (minimum gap 0.038), success vs anneal time (0.29 at $T = 10$, 0.89 at $T = 1000$), and the distribution of read energies.

4 How D-Wave is used in this project

4.1 The path of one QPU call

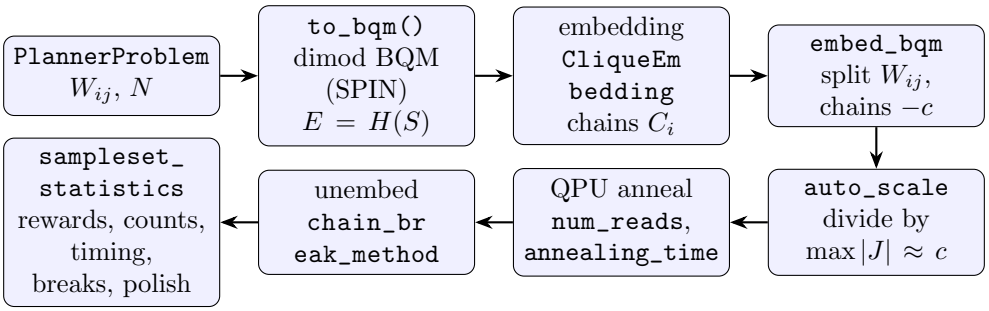


Figure 30: One call of `AnnealBackend.sample`. The embedding and scaling happen inside `FixedE mbeddingComposite` and `DWaveSampler`; unembedding and chain-break resolution happen on the way back.

4.2 Account and access: Leap

The QPUs are in D-Wave's cloud, *Leap*. Cineca grants QPU time through a Leap project; access needs only an API *token* for that project and outbound HTTPS, from any machine (Leonardo login node, the serial partition, or a laptop). Command-line tools from `dwave-cloud-client`:

```
dwave config create     # store the token in ~/.config/dwave/dwave.conf
dwave solvers -la       # list QPUs: Advantage_* (Pegasus), Advantage2_* (Zephyr)
dwave ping              # submit a tiny problem (a few ms of QPU time)
```

Getting the token, step by step (details in `Dimod/LEAP_TOKEN.md`). A free Leap account has no usable token: the token belongs to a Leap *project* with QPU time, which Cineca grants.

1. Create a Leap account at https://cloud.dwavesys.com/leap/ with your **institutional** e-mail; every user needs one.
2. Apply to Cineca's **ISCRA-C** call for D-Wave time. List the institutional e-mails of the Leap accounts and the minutes per month (Cineca's page states at most 20 min/month for 3 months, evaluated monthly). The `full` campaign needs about 124 s $\approx$ 2 min, so 5–10 min/month is ample.
3. After approval your account joins a Leap project with the minutes. Select it in Leap (username menu $\rightarrow$ Projects): each project has its own token.
4. Copy the token from the dashboard (left column, *API Token*, COPY), or from a terminal: `dwave auth login --oob` (open the printed link on your laptop, paste back the code), `dwave leap project ls`, then `dwave leap project token --project <NAME>`.
5. Store it on every machine that calls the QPU: `dwave setup --oob --project <NAME>`, or `dwave config create` (paste it), or `export DWAVE_API_TOKEN=...`. Check with `dwave solvers -la` and `main_simulation.py qpu --check`.

Never commit the token. If it leaks, RESET it on the Leap dashboard.

4.3 Functions and parameters, and their role here

function / object	what it does	parameters and settings used here
`dimod.BinaryQuadraticModel` (via `PlannerProblem.to_bqm`)	the problem in Ising form: linear h, quadratic J, offset; SPIN variables	$h = 0$, $J_{ij} = -W_{ij}$ $(i < j)$, so the BQM energy is exactly $H(S)$
`dwave.system.DWaveSampler`	connection to one QPU solver; exposes `properties`, `parameters`, `to_networkx_graph()` (working qubits and couplers)	`solver={'topology__type': 'zephyr'}` or `'pegasus'` chooses the family; `--solver NAME` pins a specific QPU
`to_networkx_graph()`	the real hardware graph, with dead qubits removed	the embedding is computed on it, so it fits the chip used
`minorminer.busclique.find_clique_embedding`	native all-to-all embedding: chains = one vertical + one horizontal line segment	`use_cache=True`; result stored in `embeddings/<solver>/clique/N...json`
`busgraph_cache(G).largest_clique()`	largest K_N the chip supports	larger N are skipped automatically; printed by `qpu --check`
`minorminer.find_embedding`	general heuristic embedding	baseline only (`qpu:zephyr:minorminer`)
`dwave.system.FixedEmbeddingComposite`	embeds the BQM with a given embedding, samples, unembeds	used for both topologies, so the two runs differ only in hardware
`chain_strength`	strength c of the couplers inside chains	`partial(uniform_torque_compensation, prefactor=p)`, $c = p\,J_{\mathrm{rms}}\sqrt{\bar{d}} \approx p$ for SK; p = `chain_strength_prefactor`, default 1.414 (§6)
`chain_break_method`	how a broken chain is read	`majority_vote` (default), `minimize_energy` (wrapped in a named function, see §8), `weighted_random`, `discard`
`return_embedding=True`	puts the embedding context (incl. the chain strength used) in `sampleset.info`	recorded in every result file
`num_reads`	anneal–readout cycles per call	1000 (`full`); sets the statistics of p_{success} per read and the QPU time
`annealing_time`	duration of one anneal, μs	20; longer is closer to adiabatic, but costs more per read
`auto_scale` (default on)	divides h, J to fit the programmable range $[-1, 1]$	the reason for the compression in §6
`SampleSet`	reads: `record.sample`, `record.energy`, `num_occurrences`, `chain_break_fraction`, `info['timing']`	`aggregate()` merges identical reads; rewards recomputed from the spins
`info['timing']`	`qpu_access_time`, `qpu_programming_time`, `qpu_sampling_time`, per-sample anneal/readout/delay (μs)	`qpu_access_time` is what Leap charges; logged per call
`solver.estimate_qpu_access_time`	D-Wave's own timing model	used by `qpu --check`, spends nothing
`dwave.embedding.embed_bqm`	the embedding step itself	used in `noise_and_chains` to measure the programmed couplings
`dwave.samplers.SimulatedAnnealingSampler`, `TabuSampler`, `SteepestDescentSolver`	classical samplers with the dimod interface	classical baselines; `sim:*` back-ends
`dimod.StructureComposite`	restricts a sampler to a hardware graph	`sim:zephyr:clique`: the same embedded problem solved by SA, no token
`dwave.system.testing.MockDWaveSampler`	offline stand-in for a QPU with a given topology	tests of the whole QPU path

function / object	what it does	parameters and settings used here
`dwave.preprocessing.SpinReversalTransformComposite`	gauge transforms	mitigation of systematic biases (§7); not yet wired into the back-ends

4.4 The back-ends of the campaign

A back-end is a string `device:topology:strategy`:

- `qpu:zephyr:clique`: Advantage2, Zephyr, degree 20, clique chains $\approx N/16 + 2$;
- `qpu:pegasus:clique`: Advantage, Pegasus, degree 15, chains $\approx N/12 + 2$;
- `sim:*`: classical stand-ins.

Both QPU back-ends solve the same instances with the same `AnnealSettings`. `analyze` writes `backend_comparison.csv` with the winner per (problem class, N): higher success per read, ties broken by lower residual.

4.5 Why the QPU budget doubles with two topologies

A QPU call is one problem on one machine. The comparison needs *every* instance solved on *both* machines:

$$n_{\text{call}}^{\text{per back-end}} = |N_s| \cdot n_{\text{classes}} \cdot n_{\text{inst}} = 20 \cdot 4 \cdot 5 = 400. \tag{9}$$

Nothing can be shared between the two back-ends. Zephyr and Pegasus are different QPUs, the embeddings differ, and each call reprograms its own chip. The time of one call is programming plus one anneal–readout–delay cycle per read:

$$t_{\text{call}} \approx t_{\text{prog}} + n_{\text{reads}}(t_a + t_{\text{ro}} + t_d) = 15\,\text{ms} + 1000 \cdot (20 + 100 + 20)\,\mu\text{s} = 15\,\text{ms} + 140\,\text{ms} = 155\,\text{ms}. \tag{10}$$

Hence

$$T = 2 \cdot 400 \cdot 0.155\,\text{s} = 2 \cdot 62\,\text{s} = 124\,\text{s}. \tag{11}$$

These are typical values: readout dominates and differs between systems, and `qpu --check` gives each solver's own estimate. To spend less, keep the comparison fair and cut what both share: fewer reads (time is almost linear in `num_reads`), fewer instances, a coarser N grid, or one topology only (`init --backends qpu:zephyr:clique`).

5 Embedding: all-to-all on Zephyr and Pegasus

SK couples every pair of spins (K_N), but each qubit has only 15 (Pegasus) or 20 (Zephyr) couplers. Each spin becomes a **chain** of qubits.

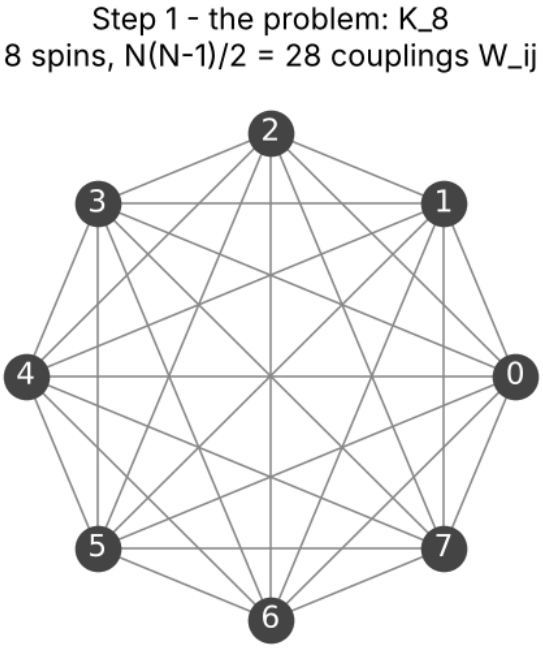


Figure 31: The problem graph: $N(N-1)/2$ couplings.

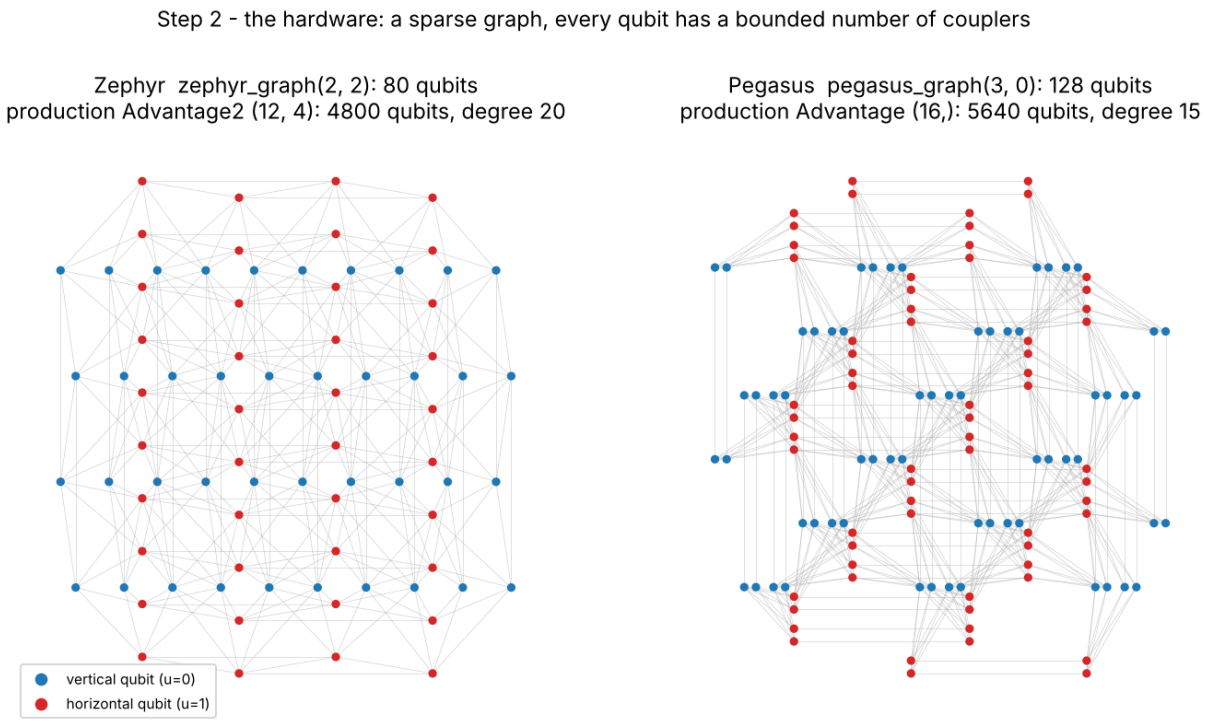


Figure 32: Qubits of two orientations (vertical, horizontal).

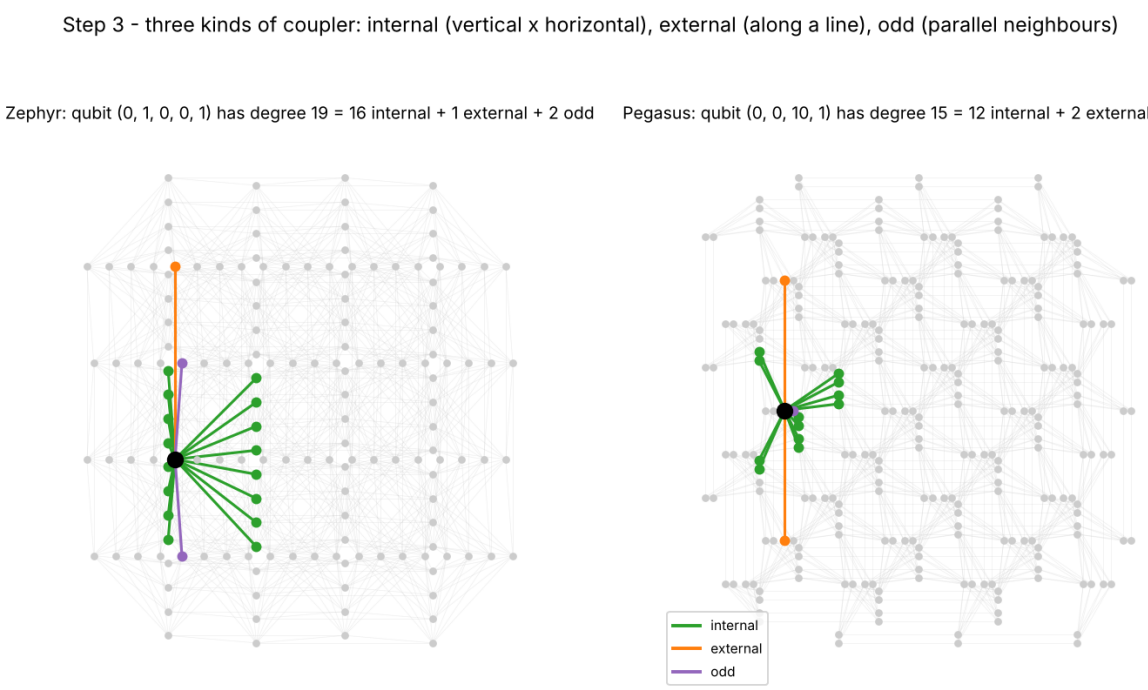


Figure 33: Three kinds of coupler: internal (vertical $\times$ horizontal), external (along a line), odd. Degree $16 + 2 + 2 = 20$ on Zephyr, $12 + 2 + 1 = 15$ on Pegasus.

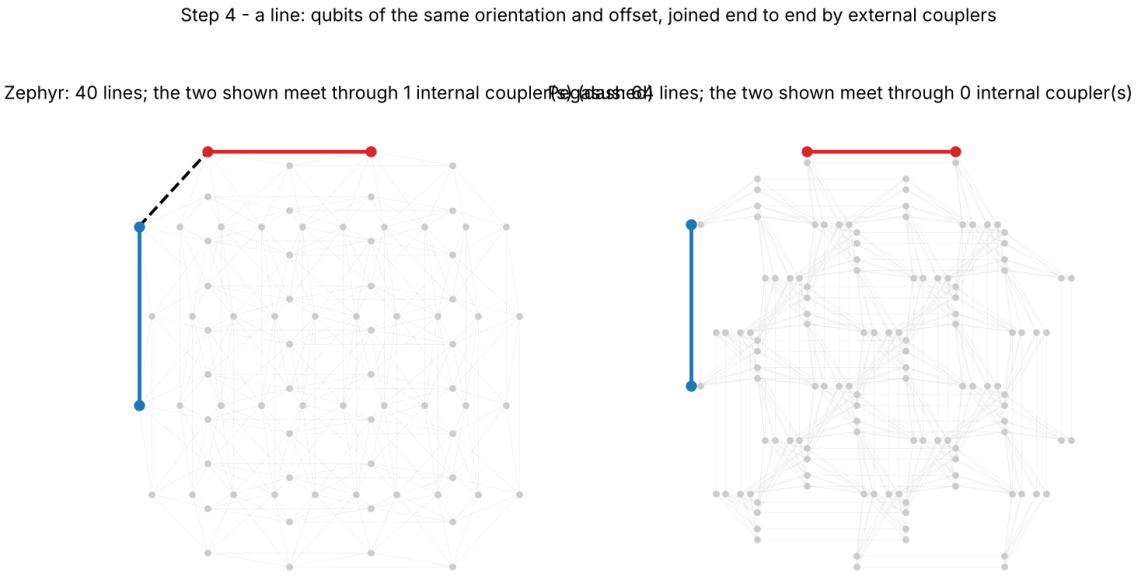


Figure 34: Qubits joined end to end by external couplers form a line.

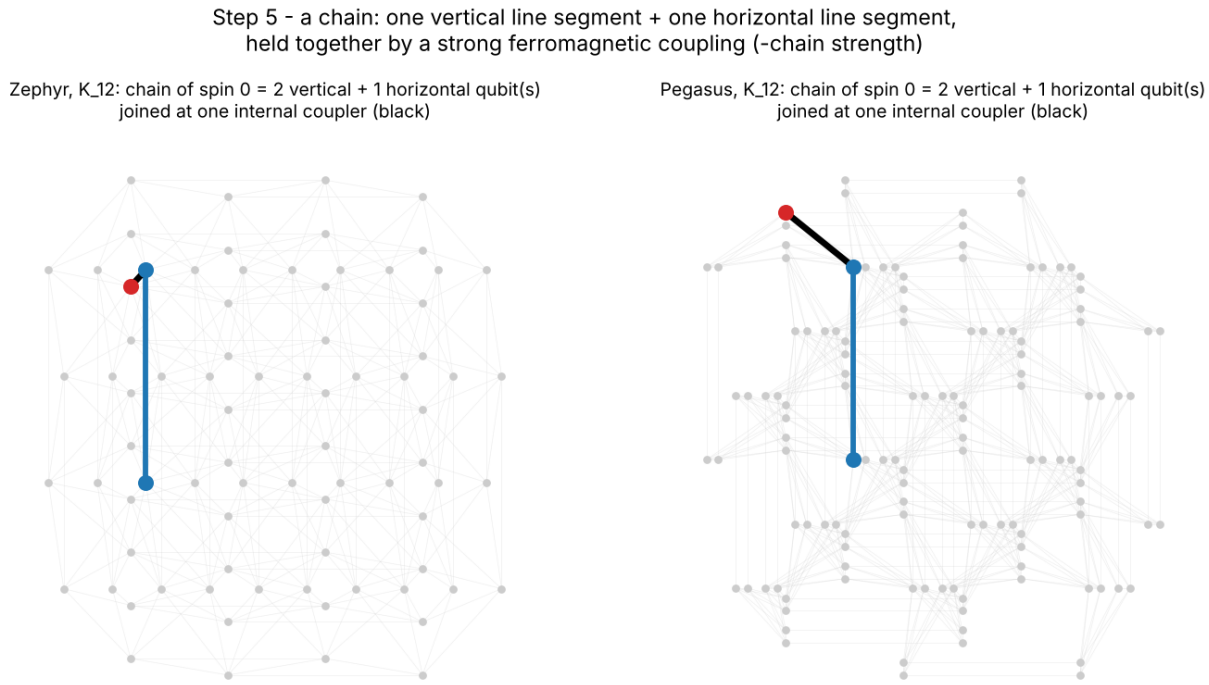


Figure 35: A chain is one vertical line segment plus one horizontal segment, joined by a single internal coupler (checked for all 160 chains at $N = 160$ on both topologies).

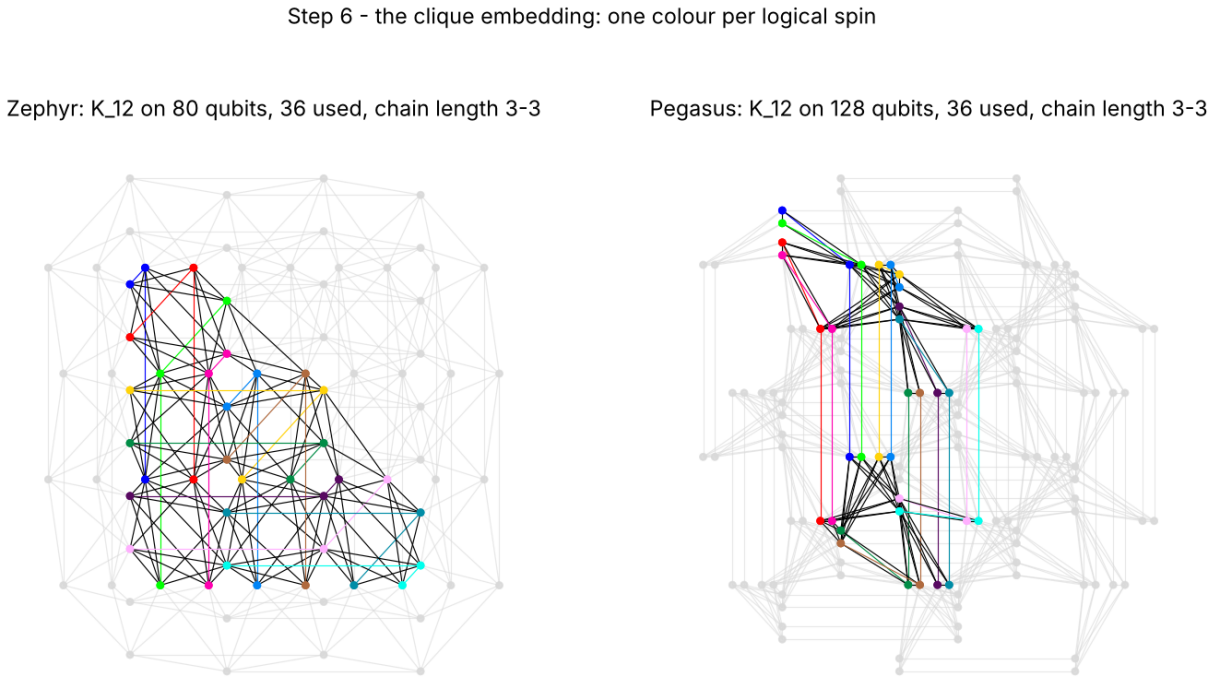


Figure 36: The full clique embedding on a small graph.

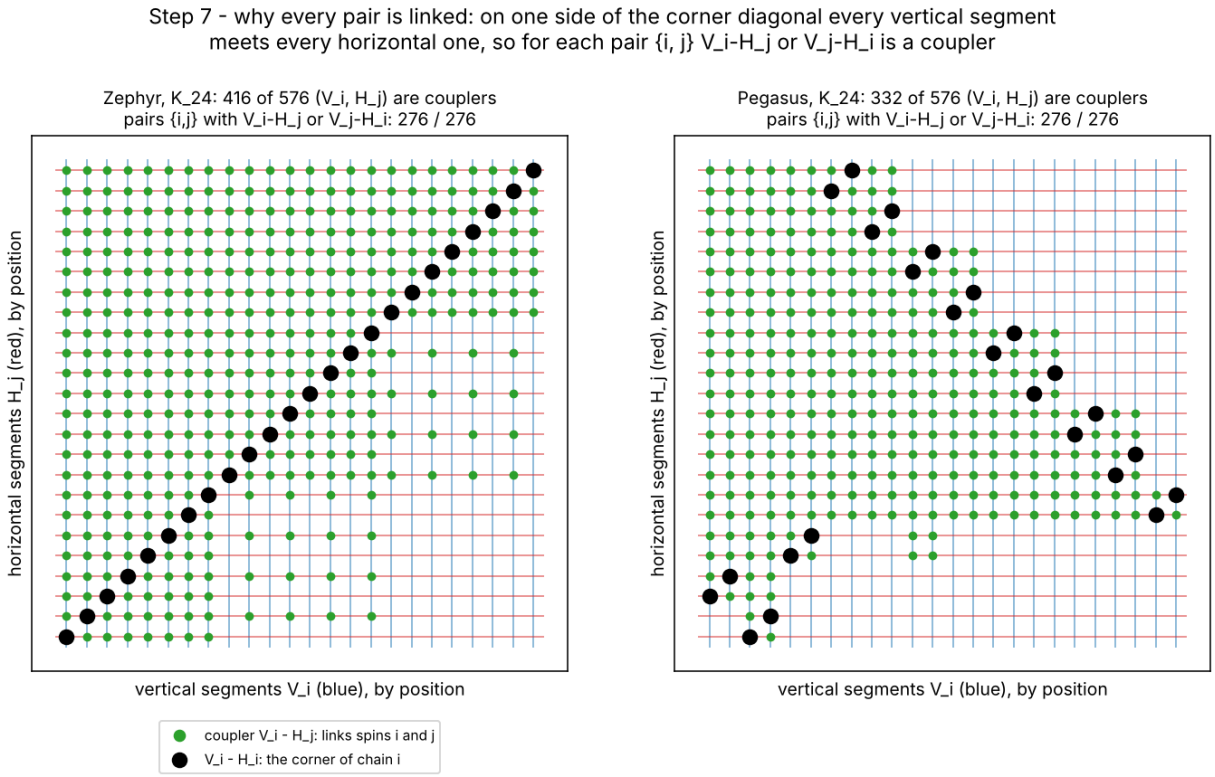


Figure 37: Why it is all-to-all: on one side of the staircase of corners, every vertical segment meets every horizontal one, so each pair $\{i, j\}$ has a V_i–H_j or V_j–H_i coupler.

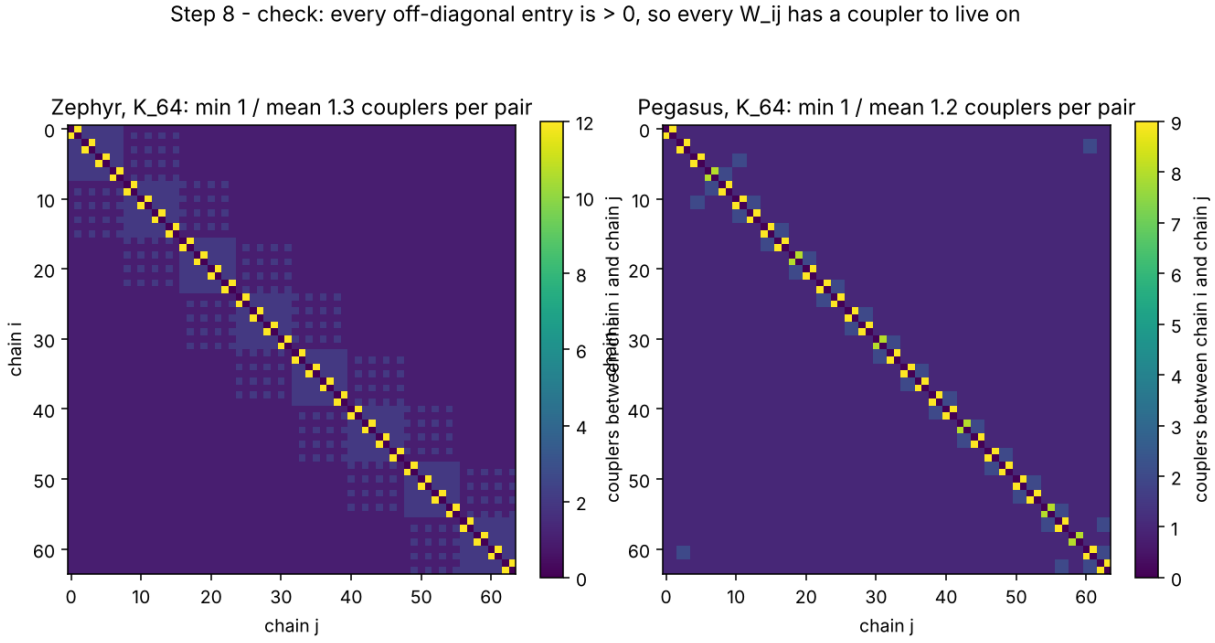


Figure 38: Couplers between every pair of chains, K_{64}: all off-diagonal entries are ≥ 1.

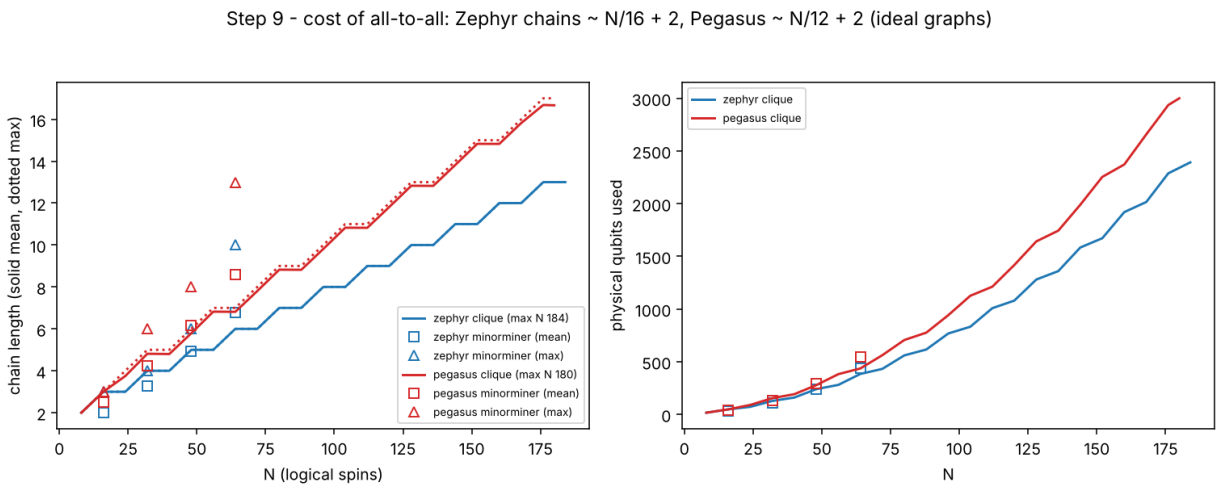


Figure 39: Chain length and qubits vs N. At $N = 160$: Zephyr 12, Pegasus 15. The heuristic embedder gives longer, uneven chains.

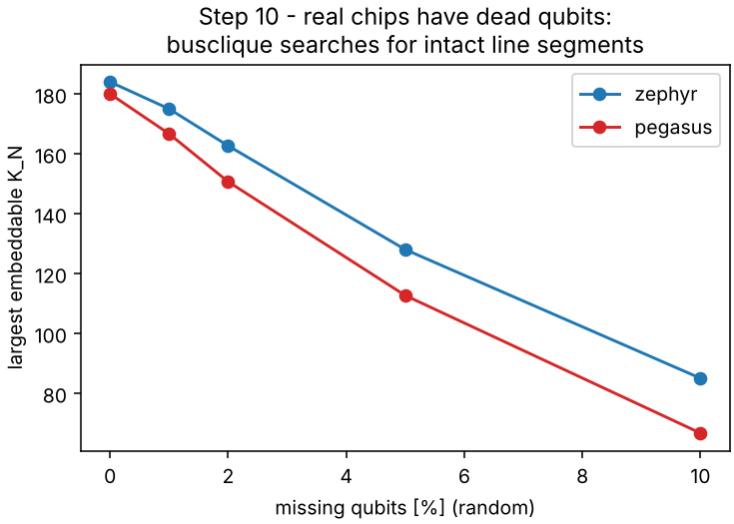


Figure 40: Missing qubits shrink the largest clique (here with random defects); Zephyr keeps its lead.

The algorithm (busclique).

1. Group qubits into lines.
2. Keep only intact segments (all qubits and external couplers working).
3. In a block of the chip, pair N vertical segments with N horizontal ones along a staircase of corners.
4. The chains are $C_i = V_i \cup H_i$.
5. Optimise the placement by dynamic programming, for the largest N and the shortest chains.
6. Cache the result.

The heuristic `find_embedding` (Cai–Macready–Roy) works differently. It routes each chain as a tree, temporarily allowing shared qubits with an exponential penalty, and keeps re-routing until no qubit is shared.

6 Chains and chain strength: how it was worked out

This section reconstructs the reasoning in the order it actually happened. Every number is reproduced by `noise_and_chains.py`.

1. The question. Inside a chain the couplers are set to $-c$. How large is c by default, and what does that do to the rest of the problem?

2. Read the function. `inspect.getsource(uniform_torque_compensation)` shows that the default is

$$c = p \cdot J_{\text{rms}} \cdot \sqrt{\bar{d}}, \qquad J_{\text{rms}} = \sqrt{\tfrac{1}{|E|}\textstyle\sum J_{ij}^2}, \quad \bar{d} = \text{average degree}, \quad p = 1.414. \tag{12}$$

The docstring's argument: half a chain feels a random torque from its neighbours, a sum of about $\bar{d}/2$ couplings of random sign, of size $\propto J_{\text{rms}}\sqrt{\bar{d}}$. The chain coupling must exceed it.

3. Evaluate it for sk. $J_{\text{rms}} = 1/\sqrt{N}$ (entries $\mathcal{N}(0, 1/N)$) and $\bar{d} = N - 1$ (complete graph), so

$$c = p\frac{1}{\sqrt{N}}\sqrt{N-1} = p\sqrt{1 - 1/N} \to p \approx 1.4. \tag{13}$$

Checked against the library on four problem classes. The script asserts that my formula and the library agree.

4. Meaning. $c/p = J_{\text{rms}}\sqrt{N-1}$ is the standard deviation of a local field h_i (measured 0.991 at $N = 160$). **The chain strength is p times the typical pull of the other agents on one agent.**

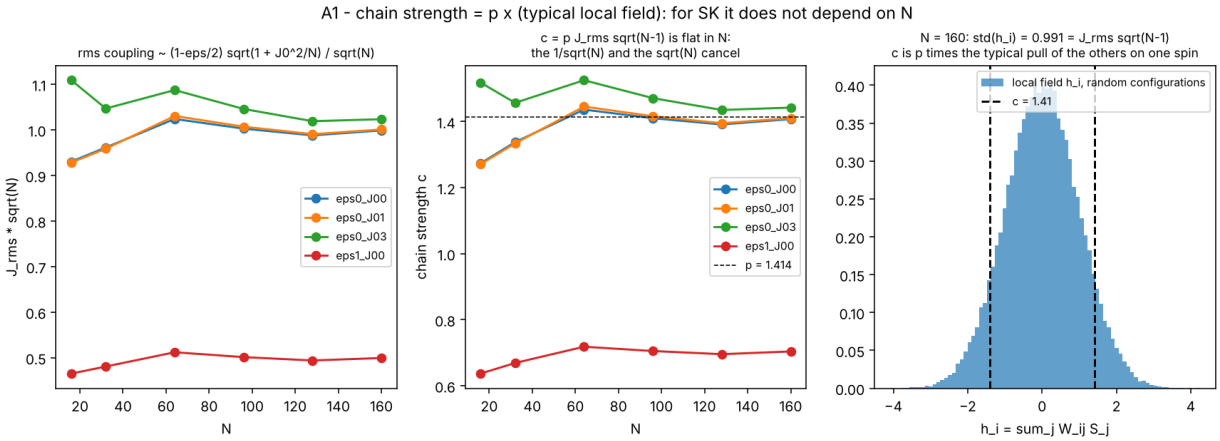


Figure 41: $J_{\text{rms}}\sqrt{N} \approx 1$; c is flat in N (≈ 0.7 at $\varepsilon = 1$, where W is halved); the local-field distribution with $\pm c$.

5. Embedding splits the couplings. W_{ij} is divided evenly over the $|E_{ij}|$ couplers between two chains. On average $|E_{ij}| = 1.21$ (Zephyr) and 1.11 (Pegasus) at $N = 160$, which reduces the rms by a factor 0.895 on Zephyr.

6. Auto-scaling divides by c. The chain couplers (1.398) are much larger than any logical one ($\max|W| = 0.29$), so $\lambda = 1/c = 0.715$.

7. Multiply.

$$\text{Zephyr, } N = 160: \quad 0.0784 \times 0.895 \times 0.715 = 0.0702 \times 0.715 = 0.0502, \tag{14}$$

$$\text{in general:} \quad \frac{J_{\text{rms}}}{c} = \frac{1}{p\sqrt{N-1}} = \frac{1}{1.414 \cdot 12.61} = 0.056 \quad (N = 160). \tag{15}$$

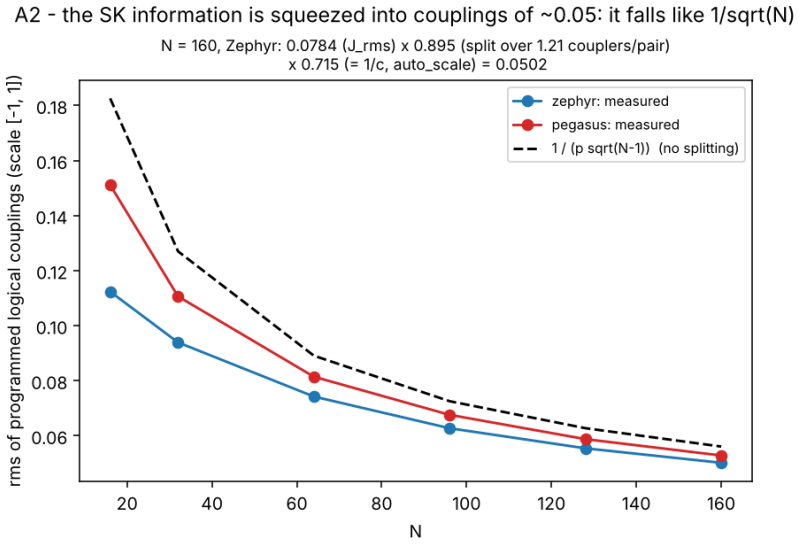


Figure 42: The programmed rms of the SK couplings vs N on both topologies, against $1/(p\sqrt{N-1})$.

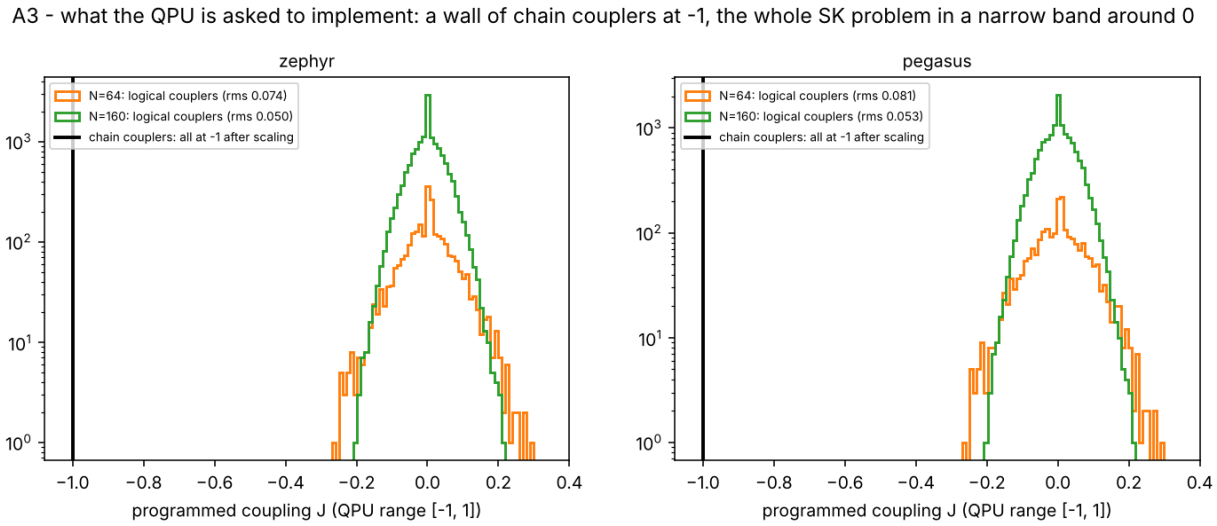


Figure 43: What the QPU receives: every chain coupler at -1, the whole instance in a band of rms 0.05 around 0.

7 Noise: what it does, what problems it raises, how to reduce it

Important. No QPU has been run in this project yet: there was no token in the development environment. The experiments below are *simulations*. Each one isolates a single mechanism with a deliberately simple model, so they show the *kind* of problem and the *direction* of each remedy, not the QPU's numbers. What I learned from them is stated with that limitation.

7.1 Random errors on the couplings

Model. Each coupler is off by $\delta \sim \mathcal{N}(0, \sigma)$ in QPU units, i.e. σc on every W_{ij}. The noisy problem is then solved exactly.

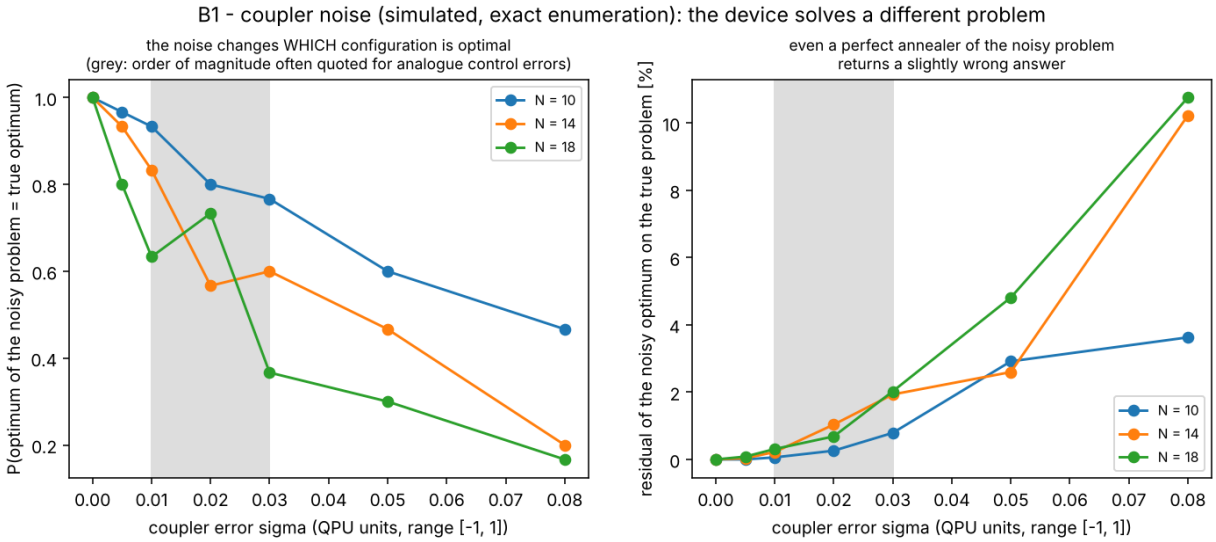


Figure 44: At $\sigma = 0.03$ the true optimum survives in 77%, 60%, 37% of instances for $N = 10, 14, 18$. Even a perfect solver of the noisy problem returns a configuration 1–2% above the minimum.

What it produces. The device solves a *different* problem. **The problem it raises.** The error is invisible from inside: reads are consistent with each other, and repeating the call does not help. **Why it grows with N.** Relative to a typical coupling the error is

$$\frac{\sigma c}{J_{\text{rms}}} = \sigma p\sqrt{N-1}; \qquad N = 160,\ \sigma = 0.02: \ 0.02 \cdot 1.414 \cdot 12.61 = 0.36. \tag{16}$$

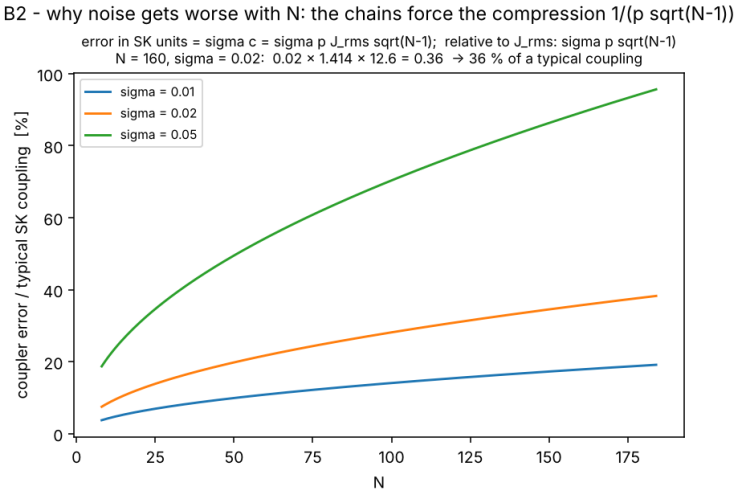


Figure 45: The compression forced by the chains turns a fixed analogue error into a relative error $\propto \sqrt{N}$.

7.2 Chain breaks, and chains that are too rigid

Model. The embedded problem on ideal Zephyr, solved by classical SA as a stand-in, scanning p.

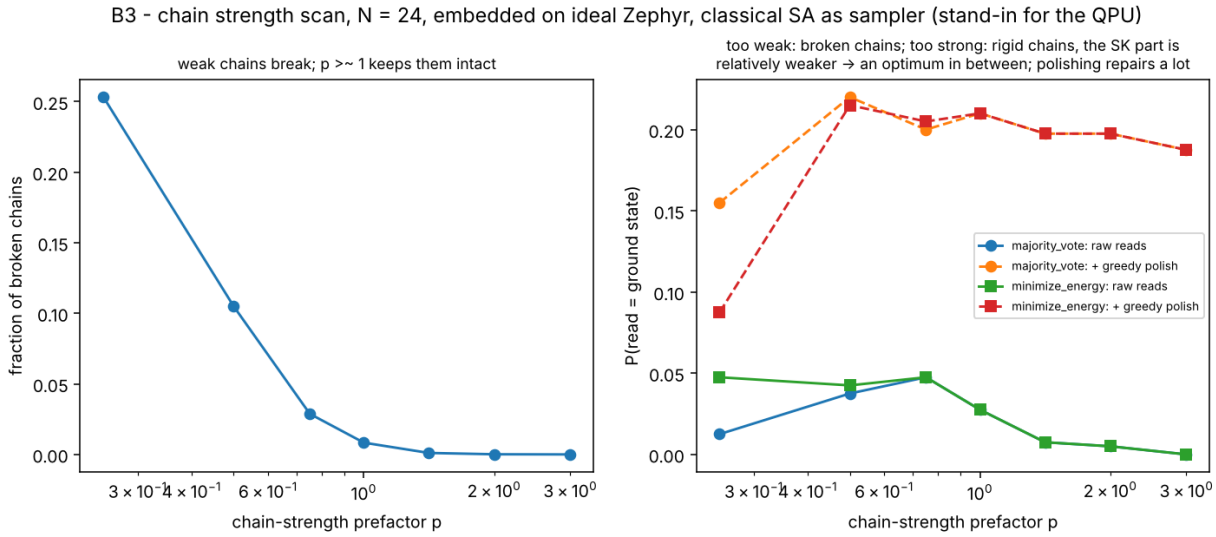


Figure 46: Breaks: 25% of chains at $p = 0.25$, below 1% for $p \geq 1$. The raw success peaks at intermediate p and falls to 0 at $p = 3$, because rigid chains freeze single flips. Greedy polishing lifts the success from ≤ 0.05 to ≈ 0.2.

Problem. Two opposite failure modes, so the best p depends on the instance. **Remedies.** Scan $p \in [0.5, 2]$; use `minimize_energy` where chains break; polish the reads; prefer shorter chains (Zephyr).

7.3 Finite effective temperature

Model. Reads drawn from $P(S) \propto e^{-\beta_{\text{eff}} H(S)/c}$. The $1/c$ is the auto-scaling, which raises the effective temperature by c.

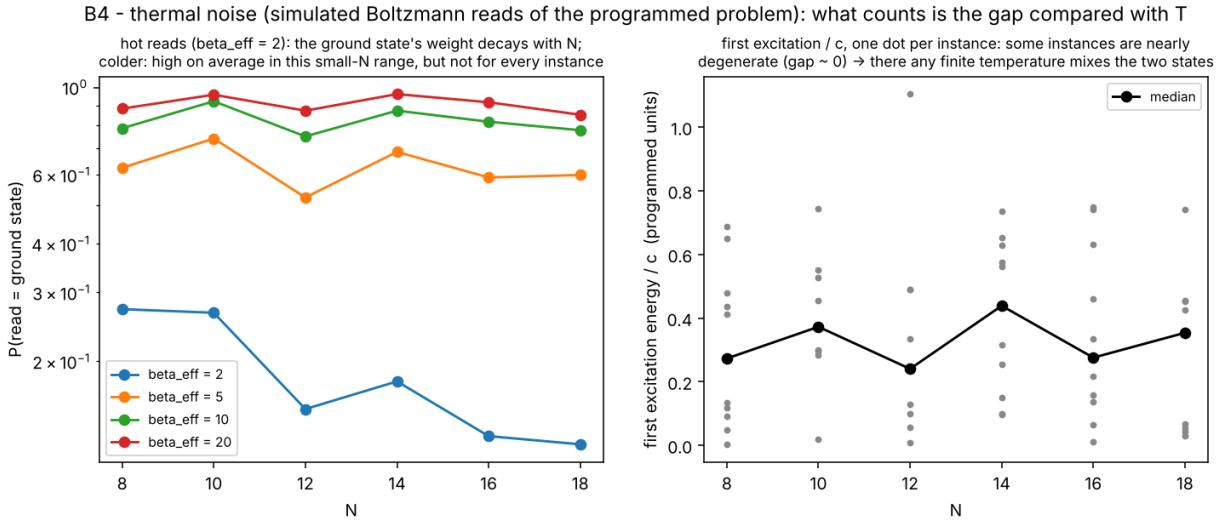


Figure 47: Hot reads: success falls from 0.27 to 0.11 between $N = 8$ and 18. Per instance, some gaps are almost zero: there any temperature splits the reads.

Remedies. Longer `annealing_time`; many reads (judge by TTS_{99}); polishing.

7.4 Systematic biases and spin-reversal transforms

Model. A fixed field offset per qubit, the same in every call. A gauge $g \in \{\pm 1\}^N$ programs $W_{ij} g_i g_j$ and maps back $S_i = g_i S'_i$: the problem is unchanged, but the offset changes sign pattern.

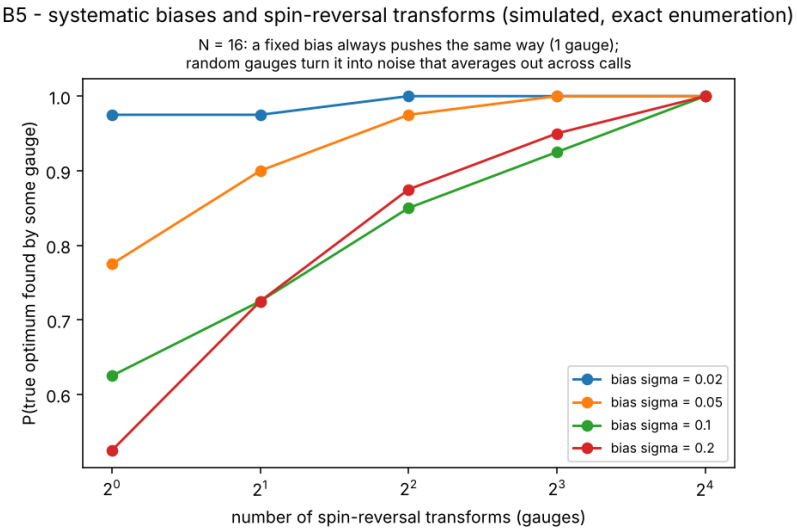


Figure 48: Bias $\sigma = 0.2$: the optimum is found with one gauge in 52% of instances, with 2/4/8/16 gauges in 72/88/95/100%.

7.5 Summary

noise source	symptom	remedy (knob)
coupler errors	a reproducible but suboptimal best read; worse as $\sqrt{N}$	shorter chains (Zephyr); smaller `chain_strength_prefactor` if chains allow it; polishing
chain breaks	`chain_break_fraction` above a few %	scan the prefactor; `minimize_energy`; polishing
rigid chains	low success with zero breaks	lower the prefactor
temperature	success per read falls with N, varies between instances	longer `annealing_time`; more `num_reads`; polishing
systematic bias	results depend on the call	spin-reversal transforms, several calls
too-fast anneal	success grows with `annealing_time`	scan the anneal time, compare TTS_{99}

8 How everything was tested, and what testing taught

8.1 The testing procedure

1. **Unit and theory tests** (`pytest`, 107 tests). They cover the model (ensemble moments, limits, recorders, fixed points), the theory (13 symbolic checks, T_c, complexity vs brute force, DMFT vs simulation), the optimisers (exact is exact, heuristics reach the optimum on small instances, SDP bound, dimod wrappers), the output registry, and two new files:

   - `test_embedding.py`: clique embeddings valid on both topologies; Zephyr chains shorter at production size; cache round trip; back-end spec parsing; the embedded simulator finds the exact ground state, with the chain strength equal to the formula;
   - `test_campaign.py`: regimes; full read statistics; the whole campaign end to end with resume; both QPU back-ends against `MockDWaveSampler` (D-Wave's offline mock QPU) with the comparison output.

2. **QPU path without a token.** `MockDWaveSampler` with Zephyr and Pegasus topologies goes through embedding on the mock's graph, chain strength, unembedding, timing fields and chain-break fraction.

3. **End-to-end runs.** The `smoke` and `small` presets with the simulated back-ends, the notebook executed headless, and `local/run_pipeline.sh` (20 s and 3 min).

4. **Environment.** The wheelhouse was built for Python 3.11 and 3.13 and installed with `--no-index` (nothing downloaded), after which the test suite passed. `local/setup_env.sh` was run on a clean copy.

5. **Figures as tests.** Every figure was looked at against the claim in its caption. Where they disagreed, the code or the claim was fixed (next subsection).

8.2 What testing found (experience)

- **Seed sent to the QPU.** The original dimod wrapper always passed `seed`. The QPU rejects unknown parameters server-side, not with a Python `TypeError`, so the existing fallback would never have triggered. Fix: pass `seed` only to samplers that declare it.

- `MinimizeEnergy` **has no** `__name__`. `unembed_sampleset` needs a named function. Found by the parametrised test; fixed with a wrapper.

- **The simulator lost the topology metadata.** A graph rebuilt from the edges of `StructureComposite` lacks the `family` attribute that busclique needs. Fix: keep the ideal graph object itself.

- **A claim that was too strong.** I first wrote that "every vertical segment crosses every horizontal one". The crossbar figure showed that this holds only on one side of the staircase of corners, which is enough for all-to-all. The text was corrected to what the data show.

- **Measuring the wrong thing.** The first SA success curve was non-monotonic because it scored the *last* state, while the solver returns the *best* state seen, polished. Re-measured properly: monotonic, 0.19→0.98.

- **Too few samples.** The ground-state energy vs N with 6 instances was dominated by sample-to-sample fluctuations. With 40 instances it follows the known finite-size law.

- **Mixing two sources of randomness.** The first gauge experiment redrew the bias for every number of gauges, giving a non-monotonic curve. A systematic bias must be *fixed*; with that, the expected monotonic gain appears.

- **A confusing representation.** Fixing the global sign $S \to -S$ at every time step made whole columns flip colour. One sign per trajectory fixed it.

The general lesson: in this problem many plausible-looking curves are artefacts of what is measured (last vs best state, per-step vs per-run sign, redrawn vs fixed noise). Each figure needs a check against an independent prediction: an exact enumeration, a formula, or a library value.

9 Running it

Locally (`README_LOCAL.md`):

```
bash local/setup_env.sh && source .venv/bin/activate
dwave config create                                  # only for the QPU
PRESET=small bash local/run_pipeline.sh              # ~3 min, simulated annealers
PRESET=full ANNEALER=both bash local/run_pipeline.sh # production, Zephyr + Pegasus
```

On Leonardo (`README_CINECA.md`): build the wheelhouse locally (`cineca/build_wheelhouse.sh 3.11`), upload it, `cineca/install_offline.sh`, then `init`, `embed`, `qpu --check`, `cineca/submit_all.sh`. The QPU stage can also run on the laptop: `cineca/sync.sh pull-inputs`, `qpu`, `push-anneal`.

Suggested first QPU session.

1. `qpu --check`.
2. Two calls per topology (`QPU_CALLS=2`).
3. A chain-strength scan $p \in \{0.5, 0.75, 1, 1.41, 2\} \times \{$`majority_vote`, `minimize_energy`$\}$ on 2–3 instances at the N where success starts to drop.
4. An `annealing_time` scan.
5. The full campaign with the chosen settings.

A Glossary

term	meaning
spin, decision S_i	the binary choice of agent i
coupling J_{ij}, W_{ij}	effect of j on i's reward; symmetric part seen by the planner
local field h_i	$\sum_j W_{ij} S_j$, the pull of the others
satisfaction λ_i	$S_i h_i$; a flip of i changes the energy by $2\lambda_i$
energy H, reward R_N	$H = -\frac{1}{2}\sum \lambda_i$, $R_N = -2H/N$
local minimum	every $\lambda_i \geq 0$; fixed point of the zero-temperature game at $\varepsilon = 0$
ground state	minimum of H; $H/N \to -0.7632$
basin, barrier	starts that descend to a minimum; energy to climb between minima
frustration	loops of couplings that cannot all be satisfied
temperature, replica	acceptance of uphill moves; a copy of the system at one temperature
chain, chain strength c	qubits representing one spin; coupling $-c$ holding them together
auto-scale	division of the programmed problem into $[-1, 1]$ (by $\approx c$ here)
read, p_{success}, TTS_{99}	one anneal + measurement; fraction of reads at the optimum; $t_{\text{read}} \ln 0.01 / \ln(1-p)$
gauge (spin-reversal)	$S_i \to g_i S_i$ with $W_{ij} \to g_i g_j W_{ij}$: same problem, different hardware signs